CRASH COURSE PLAN — v3 (thêm Part 14–20)

React Crash Course

> Lộ trình học chi tiết từng phần — kèm giải thích cơ chế render bên trong

Repo: crash-course / react-crash-course-traversy

Tài liệu bám theo React 19.2 (phát hành 01/10/2025, bản ổn định mới nhất) + Vite + React Router — kiểm chứng lại theo react.dev & reactrouter.com (09/2026)

Part 14–20 thêm 11/2026, đã chạy thử trên react 19.3.1 · react-router 7.18 · typescript 7.0 · next 16.4

Dự án hiện tại của bạn: react 19.2.5 · react-router-dom 7.14.2 · vite 8 · tailwindcss 4

Bản v2: tài liệu HTML gốc từng bị mất, PDF cũ là nguồn duy nhất còn lại — bản này tái dựng lại từ PDF đó và audit toàn bộ theo docs chính thức hiện tại

Mục lục

JS JavaScript ES6+ cho React — NỀN TẢNG NGÔN NGỮ

00 React hoạt động thế nào (render & reconciliation) — NỀN TẢNG

01 Khởi tạo dự án với Vite & cấu trúc thư mục

02 JSX, Component & Props

03 State với useState — tư duy bất biến

04 Render danh sách, key & render có điều kiện

05 Sự kiện & Form (controlled component)

06 useEffect — dùng đúng và dùng ít

07 Gọi API & Custom Hook

08 Chia sẻ state: lifting up, Context, useReducer

09 React Router (routing, loader, form action)

10 Hiệu năng: re-render, memo & React Compiler

11 React 19: Actions, hooks mới & use()

12 Kết nối API thật (Laravel / NestJS) & build production

13 Testing với Vitest + Testing Library — TUỲ CHỌN

14 TypeScript cho React — MỚI

15 TanStack Query thực chiến: cache, phân trang, mutation — MỚI

16 Form với React Hook Form + Zod — MỚI

17 Đăng nhập thực chiến: axios, refresh token, protected route — MỚI

18 State toàn cục: Zustand và Redux Toolkit — MỚI

19 Next.js cơ bản và Server Components — MỚI

20 Code splitting, accessibility và checklist trước khi deploy — MỚI

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh hoạ để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Dự án xuyên suốt: App quản lý tin tuyển dụng (job listings) — danh sách, chi tiết, thêm, sửa, xoá. Đúng dự án bạn đã làm theo Traversy, nhưng lần này đi sâu vào vì sao mỗi thứ hoạt động như vậy. Ở Part 12 ta sẽ thay json-server bằng API Laravel/NestJS thật mà bạn tự viết — đó là lúc bạn thành fullstack thật sự.

Bản v3 thêm gì: Part 14–20 lấp các khoảng trống so với yêu cầu đi làm: TypeScript, TanStack Query, React Hook Form + Zod, đăng nhập với refresh token và protected route, Zustand và Redux Toolkit, Next.js, code splitting và accessibility. Code trong các Part này được lấy từ một app đã chạy **type check, 15 test và build**, và luồng đăng nhập đã chạy bằng trình duyệt thật với API NestJS thật. Các Part cũ giữ nguyên. Part 12.3 dùng `localStorage` để đơn giản hoá, còn Part 17 là cách nên dùng trong dự án thật.

Bản v2 này khác gì bản PDF gốc: file HTML gốc của tài liệu này đã bị mất (không commit vào git), chỉ còn PDF 39 trang — bản này tái dựng lại toàn bộ nội dung từ PDF đó, rồi audit từng phần theo docs chính thức hiện tại: `react.dev/learn`, `react.dev/reference/react`, `react.dev/reference/react-dom`, `react.dev/blog` (bài React 19, React 19.2, React Compiler v1.0), và `reactrouter.com`. Các chỗ đã sửa: (1) API xử lý lỗi của React Router — bản cũ dùng `errorElement` + `throw new Response()`, docs hiện tại khuyến nghị `ErrorBoundary` + `useRouteError` + `data()`; (2) import Data mode nên lấy từ `"react-router"` thay vì `"react-router-dom"`; (3) bổ sung ngày phát hành chính xác cho React 19.2 (01/10/2025) và React Compiler v1.0 (07/10/2025). Thêm 4 mục "so sánh 2 cách" mới (đánh dấu khung xanh): 7.4, 8.4, 9.1 (nâng cấp), 11.9.

PART J — NỀN TẢNG NGÔN NGỮ

JavaScript ES6+ cho React

> **Mục tiêu:** Nắm chắc các tính năng JS mà React dùng ở mọi dòng code. Phần lớn lỗi "React khó hiểu" thực chất là lỗi chưa vững JS — đặc biệt là tham chiếu vs giá trị, closure và bất biến (immutability).

J.1 let / const và scope

```
const user = { name: 'Nhat' };              // const = KHÔNG gán lại được biến
user.name = 'Long';                           // vẫn sửa được thuộc tính bên trong!
// user = {}                    ✘ TypeError: Assignment to constant variable

let count = 0; count++;                      // let khi cần gán lại
// var: KHÔNG dùng nữa (scope theo function, bị hoisting khó lường)
```

Quy tắc: mặc định dùng `const`, chỉ đổi sang `let` khi thật sự cần gán lại. Trong React bạn gần như chỉ dùng `const`.

J.2 Tham chiếu vs giá trị — gốc rễ của mọi bug React

```
// Primitive (number, string, boolean, null, undefined) → so sánh theo GIÁ TRỊ
1 === 1                   // true
'a' === 'a'               // true

// Object / Array / Function → so sánh theo THAM CHIẾU (địa chỉ ô nhớ)
[] === []                  // false !!
{} === {}                  // false !!
const a = [1]; const b = a; a === b     // true (cùng 1 địa chỉ)
```

Vì sao cực kỳ quan trọng: React quyết định "có cần render lại không" bằng phép so sánh `Object.is` (gần như `===`). Nên: (1) sửa trực tiếp mảng/object trong state → React thấy vẫn cùng tham chiếu → không render lại; (2) truyền `{}` hay `[]` mới mỗi lần render vào props/dependency → React thấy luôn khác → render lại / chạy effect vô hạn. Hai lỗi này chiếm phần lớn bug của người mới.

J.3 Destructuring, spread & rest

```
// Destructuring object — cách nhận props chuẩn của React
const { title, salary = 'Thoả thuận' } = job;      // có giá trị mặc định
const { company: { name } } = job;                  // lồng nhau

// Destructuring array — cách useState trả về
const [count, setCount] = useState(0);

// Spread — TẠO BẢN SAO MỚI (cốt lõi của immutability)
const updated  = { ...job, title: 'Mới' };        // copy object + ghi đè 1 field
const appended = [...jobs, newJob];                 // copy mảng + thêm phần tử

// Rest — gom phần còn lại
const { id, ...rest } = job;
function Button({ children, ...props }) { }         // pattern rất hay dùng
```

Spread chỉ copy NÔNG (shallow): `{...job}` copy được field cấp 1, nhưng `job.company` vẫn là cùng một object. Muốn sửa dữ liệu lồng nhau phải spread từng cấp: `{ ...job, company: { ...job.company, name: 'X' } }`.

J.4 Arrow function & this

```
const double = (x) => x * 2;                // return ngầm
const make   = (x) => ({ value: x });       // trả object → PHẢI bọc ()
const sum    = (a, b) => { return a + b; };  // có {} thì phải return tường minh
```

Arrow function không có `this` riêng — nó mượn `this` của scope bao ngoài. Với React function component (kiểu hiện đại) bạn gần như không đụng tới `this` nữa; chỉ cần nhớ điều này khi đọc code class component cũ.

J.5 Các method mảng phải thuộc lòng

Method	Trả về	Dùng trong React để
`map()`	Mảng mới cùng độ dài	Render danh sách — dùng nhiều nhất
`filter()`	Mảng mới ít phần tử hơn	Xoá item khỏi state, lọc kết quả
`find()`	1 phần tử hoặc `undefined`	Tìm item theo id
`some()` / `every()`	boolean	Kiểm tra điều kiện tồn tại / toàn bộ
`reduce()`	Giá trị gộp bất kỳ	Tính tổng, gom nhóm
`slice()`	Mảng mới	An toàn — copy một đoạn
`splice()`, `push`, `sort`, `reverse`	Sửa tại chỗ	KHÔNG dùng trực tiếp trên state — phải copy trước

```
// ✘ SAI — sửa tại chỗ, React không thấy thay đổi
jobs.push(newJob); setJobs(jobs);
jobs.sort((a, b) => a.title.localeCompare(b.title));

// ✔ ĐÚNG — luôn tạo mảng mới
setJobs([...jobs, newJob]);
setJobs([...jobs].sort((a, b) => a.title.localeCompare(b.title)));
setJobs(jobs.filter((j) => j.id !== id));
```

J.6 Toán tử hay dùng trong JSX

```
{ isLoading && <Spinner /> }                    // short-circuit: true thì render
{ jobs.length > 0 ? <List /> : <Empty /> }        // ternary: if/else
{ job?.company?.name }                           // optional chaining: null thì undefined
{ job.salary ?? 'Thoả thuận' }                   // nullish: chỉ thay khi null/undefined
```

Bẫy kinh điển: `{jobs.length && <List />}` — nếu `length === 0`, JS trả về số `0` và React sẽ in ra chữ "0" trên màn hình. Luôn ép sang boolean: `{jobs.length > 0 && ...}`. Ngoài ra: `??` khác `||` — `||` thay cả `0` và `''` (falsy), `??` chỉ thay `null` / `undefined`.

J.7 Module: import / export

```
// export mặc định — 1 cái/file, import đặt tên tuỳ ý
export default function JobCard() { }
import JobCard from './components/JobCard';

// named export — nhiều cái/file, import phải đúng tên
export const API_URL = '/api/jobs';
export function formatSalary() { }
import { API_URL, formatSalary as fmt } from './utils';
```

J.8 Bất đồng bộ: Promise, async/await, fetch

```
async function getJobs() {
  try {
    const res = await fetch('/api/jobs');

    // QUAN TRỌNG: fetch KHÔNG throw khi server trả 404/500 — phải tự kiểm tra
    if (!res.ok) throw new Error(`HTTP ${res.status}`);

    return await res.json();
  } catch (err) {
    console.error(err);
    throw err;
  }
}
```

Ba trạng thái của Promise: `pending → fulfilled / rejected`. `await` chỉ dùng được trong hàm `async`, và hàm `async` luôn trả về Promise. Đây là lý do bạn không được khai `async` cho callback của `useEffect` trực tiếp (Part 06).

J.9 Closure — nền tảng để hiểu Hooks

```
function counter() {
  let count = 0;                    // biến này "sống" nhờ closure
  return () => ++count;              // hàm trong nhớ được biến ngoài
}
const inc = counter();
inc(); inc();                              // 1, 2
```

Closure = hàm "nhớ" được môi trường nơi nó được tạo ra. React dùng closure cho toàn bộ hệ thống hook: mỗi lần render tạo ra một closure mới với giá trị state của lần render đó. Hiểu điều này là hiểu vì sao có hiện tượng "stale closure" — hàm trong `setTimeout` hay `useEffect` nhìn thấy giá trị state cũ của lần render mà nó được tạo (Part 03 & 06).

J.10 Bảng tra: JS ↔ PHP (cho người học fullstack)

JavaScript	PHP
`const x = 1`	`$x = 1;`
`obj.prop`	`$obj->prop`
`{ a: 1 }` (object)	`['a' => 1]` (associative array)
`arr.map(fn)`	`array_map($fn, $arr)`
`` `Hi ${name}` ``	`"Hi $name"`
`a ?? b`, `a?.b`	`$a ?? $b`, `$a?->b` (giống hệt)
`JSON.stringify / parse`	`json_encode / json_decode`

`npm`, `package.json`	`composer`, `composer.json`
Async mặc định (event loop)	Đồng bộ mặc định

Việc cần làm (JS thuần, chạy bằng Node hoặc console trình duyệt):

1. Chứng minh `[] === []` là false, và giải thích được vì sao điều đó khiến React render lại
2. Cho mảng 5 job, viết: lọc job có salary > 100, map lấy title, sắp xếp A→Z — không được sửa mảng gốc (kiểm tra bằng cách in mảng gốc sau đó)
3. Viết hàm cập nhật 1 job trong mảng theo id, dùng `map` + spread, giữ nguyên bất biến
4. Sửa 1 field lồng 2 cấp (`job.company.name`) mà không mutate object gốc
5. Viết `async function getJobs()` dùng `fetch` tới json-server, có kiểm tra `res.ok` và `try/catch`
6. Viết ví dụ closure của riêng bạn, giải thích được "stale closure" là gì

PART 00 — NỀN TẢNG

React hoạt động thế nào

Mục tiêu: Hiểu React thực sự làm gì trước khi viết component. React không phải framework đầy đủ như NestJS/Laravel — nó chỉ là thư viện dựng giao diện, với một ý tưởng trung tâm: **UI = f(state)**. Bạn mô tả "với state này thì giao diện trông thế nào", React lo phần cập nhật DOM.

0.1 Declarative vs Imperative

```
// Imperative (DOM thuần) — bạn ra lệnh từng bước
const el = document.createElement('div');
el.textContent = count;
document.body.appendChild(el);
// và mỗi lần count đổi, bạn phải TỰ tìm đúng node để sửa

// Declarative (React) — bạn mô tả kết quả mong muốn
return <div>{count}</div>;
// count đổi → React tự tính ra cần sửa gì trên DOM
```

0.2 Virtual DOM & Reconciliation

Mỗi lần render, component của bạn trả về một cây object mô tả UI (React element — chính là Virtual DOM). React so sánh cây mới với cây của lần render trước (**reconciliation**), tính ra tập thay đổi nhỏ nhất, rồi mới đụng vào DOM thật. DOM thật rất chậm; so sánh object trong RAM thì nhanh — đó là toàn bộ lý do Virtual DOM tồn tại.

```
   Trigger        Lần render đầu, hoặc setState được gọi
      |
      ▼
  Render phase    React gọi hàm component của bạn để lấy cây element mới.
                  Phase này phải THUẦN KHIẾT: không sửa biến ngoài, không
                  gọi API, không đụng DOM. React có thể huỷ và chạy lại
                  phase này bất cứ lúc nào
      |
      ▼
  Diff/Reconcile  So cây mới với cây cũ theo vị trí và type;
                  với danh sách thì so theo key
      |
      ▼
  Commit phase    Áp thay đổi lên DOM thật, cập nhật ref,
                  rồi chạy useEffect
      |
      ▼
  Browser paint   Trình duyệt vẽ lại màn hình
```

Hệ quả cần nhớ: "render" không đồng nghĩa với "cập nhật DOM". React có thể gọi lại hàm component của bạn nhiều lần mà DOM không đổi gì cả. Vì vậy: không bao giờ đặt side-effect (gọi API, ghi localStorage, sửa biến toàn cục) trực tiếp trong thân component.

0.3 Luồng dữ liệu một chiều

Dữ liệu chảy từ cha xuống con qua props. Con không được sửa props. Muốn con tác động lên cha thì cha truyền xuống một hàm callback. Nhờ ràng buộc này, khi UI sai bạn luôn truy ngược được nguồn dữ liệu — khác hẳn kiểu two-way binding dễ rối.

0.4 StrictMode: vì sao code chạy 2 lần

```jsx
// src/main.jsx
<StrictMode>
  <App />
</StrictMode>
```

Ở chế độ development, `StrictMode` cố ý gọi component 2 lần và chạy effect 2 lần (mount → unmount → mount lại) để phát hiện sớm: component không thuần khiết, effect thiếu cleanup. Ở production nó không làm vậy. Thấy log in 2 lần là bình thường — đừng gỡ `StrictMode`, hãy sửa code cho đúng.

0.5 React 19 & hệ sinh thái hiện tại

Thành phần	Vai trò
`react`	Lõi: component, hook, element — không biết gì về trình duyệt
`react-dom`	Renderer cho web (`createRoot` , các hook liên quan DOM/form)
Vite	Dev server + bundler (thay thế Create React App đã ngừng phát triển)
React Router	Điều hướng nhiều trang (Part 09)

React Compiler v1.0	Phát hành 07/10/2025 — trình biên dịch tự memo hoá, giảm nhu cầu `useMemo` / `useCallback` viết tay (Part 10)

Đã kiểm chứng: React 19.2 phát hành 01/10/2025 (blog chính thức `react.dev/blog/2025/10/01/react-19-2`), React Compiler v1.0 phát hành 07/10/2025 (`react.dev/blog/2025/10/07/react-compiler-1`) — cả hai đều là bản ổn định, không phải beta/RC.

Việc cần làm:

1. Chưa code — vẽ lại sơ đồ mục 0.2 ra giấy bằng ngôn từ của bạn
2. Trả lời: vì sao React cần "key" khi render danh sách (gợi ý: liên quan tới bước Diff)
3. Trả lời: "render" và "cập nhật DOM" khác nhau chỗ nào
4. Mở app hiện có, thêm `console.log` trong thân một component rồi quan sát số lần in ra — giải thích bằng StrictMode

---

PART 01

Khởi tạo dự án với Vite & cấu trúc thư mục

Mục tiêu: Hiểu Vite làm gì, mỗi file trong dự án đóng vai trò gì, và cách tổ chức thư mục để dự án lớn không loạn.

1.1 Tạo dự án

```
npm create vite@latest react-crash-course -- --template react
cd react-crash-course
npm install
npm run dev          # → http://localhost:5173
```

Script	Tác dụng
`npm run dev`	Dev server, hot reload (HMR) — giữ nguyên state khi sửa code
`npm run build`	Build production vào `dist/` (minify, tách chunk, hash tên file)
`npm run preview`	Chạy thử bản build — kiểm tra trước khi deploy
`npm run lint`	ESLint — bắt lỗi hook sai quy tắc

Vì sao Vite nhanh: ở dev, Vite không bundle — nó phục vụ file dưới dạng ES module gốc để trình duyệt tự nạp, nên khởi động gần như tức thì bất kể dự án lớn cỡ nào. Chỉ khi build nó mới bundle (bằng Rollup). Create React App (webpack) bundle lại toàn bộ mỗi lần khởi động — chậm hơn nhiều, và đã ngừng phát triển (không còn được đội React khuyến nghị).

1.2 Điểm khởi đầu

```
// index.html — Vite dùng file này làm entry (không phải file JS)
<div id="root"></div>
<script type="module" src="/src/main.jsx"></script>

// src/main.jsx
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';
import './index.css';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
);
```

Toàn bộ app React của bạn sống bên trong một thẻ `div` duy nhất — đó chính là ý nghĩa của "Single Page Application".

1.3 Cấu trúc thư mục nên theo

Thư mục	Chứa gì
`src/components/`	Component tái sử dụng, không gắn với route (Card, Navbar, Spinner)
`src/pages/`	Component tương ứng một route (HomePage, JobPage)
`src/layouts/`	Khung chung bọc nhiều trang (MainLayout: Navbar + `<Outlet />`)
`src/hooks/`	Custom hook (`useJobs`, `useFetch`) — Part 07
`src/services/` hoặc `api/`	Hàm gọi API — tách khỏi component
`src/context/`	Context provider — Part 08
`src/assets/`	Ảnh, icon (Vite tự hash khi build)

Quy ước đặt tên: file component viết PascalCase (`JobCard.jsx`), hook bắt đầu bằng `use` (`useJobs.js`), hằng số viết UPPER_SNAKE. Nhất quán quan trọng hơn "chọn đúng" — chọn một kiểu rồi giữ nguyên.

1.4 Biến môi trường

```
# .env — CHỈ biến có tiền tố VITE_ mới được lộ ra client
VITE_API_URL=http://localhost:8000/api/v1
```

```
const baseUrl = import.meta.env.VITE_API_URL;
```

Cảnh báo bảo mật: mọi biến `VITE_*` đều bị nhúng thẳng vào file JS gửi xuống trình duyệt — ai cũng xem được. Không bao giờ đặt secret key, DB password, API secret vào đây. Client chỉ được biết những gì công khai được.

Việc cần làm:

1. Tạo dự án Vite mới (hoặc dùng lại dự án job listings đang có)
2. Sắp xếp lại thư mục theo bảng 1.3 — tách `components/`, `pages/`, `layouts/`
3. Đọc kỹ `index.html` + `main.jsx`, giải thích được app "gắn" vào DOM chỗ nào
4. Tạo `.env` với `VITE_API_URL`, đọc ra và log — xác nhận nó xuất hiện trong bundle sau khi build (mở file trong `dist/` tìm chuỗi đó)
5. Chạy `npm run build` + `npm run preview`, xem kích thước bundle

PART 02

JSX, Component & Props

Mục tiêu: Viết component đúng chuẩn, hiểu JSX thực chất là gì, và nắm quy tắc truyền props.

2.1 JSX thực chất là gì

```
// Bạn viết:
const el = <h1 className="title">Xin chào</h1>;

// Vite/Babel biên dịch thành lời gọi hàm (React 17+ dùng jsx runtime):
const el = jsx('h1', { className: 'title', children: 'Xin chào' });
```

Nghĩa là JSX chỉ là cú pháp đường để tạo object mô tả UI. Vì cuối cùng là JavaScript, nên có các quy tắc:

Quy tắc	Vì sao
`className` thay `class`	`class` là từ khoá của JS
`htmlFor` thay `for`	Tương tự
Thuộc tính viết camelCase: `onClick`, `tabIndex`	Đang là thuộc tính object JS
Phải đóng mọi thẻ: `<img />`, ` `	JSX nghiêm ngặt hơn HTML
Chỉ trả về một node gốc	Một hàm chỉ `return` được 1 giá trị → dùng `<>...</>` (Fragment)
`{ }` để nhúng biểu thức	Chỉ nhận biểu thức, không nhận `if` / `for`

2.2 Component

```
// Component = HÀM trả về JSX. Tên PHẢI viết hoa chữ đầu.
export default function JobCard({ title, type, salary }) {
  return (
    <div className="rounded-xl bg-white p-4 shadow">
      <h3>{title}</h3>
      <p>{type}</p>
      <p>{salary ?? 'Thoả thuận'}</p>
    </div>
  );
}
```

Vì sao tên component phải viết hoa: JSX quy ước chữ thường (`<div>`) là thẻ HTML, chữ hoa (`<JobCard>`) là component. Đặt tên `jobCard` thì React sẽ đi tìm thẻ HTML tên "jobcard" và render ra trang trắng không báo lỗi rõ ràng.

2.3 Props

```
// Cha truyền xuống
<JobCard title={job.title} type={job.type} salary={job.salary} />
<JobCard {...job} />                        // spread khi tên trùng khớp

// Con nhận — destructure ngay ở tham số, có giá trị mặc định
function JobCard({ title, type = 'Full-Time' }) { }

// children: nội dung đặt giữa 2 thẻ
function Card({ children }) { return <div className="card">{children}</div>; }
<Card><h3>Tiêu đề</h3></Card>
```

Props là read-only. Component không được sửa props nhận vào (`props.title = 'x'` là sai). Nếu cần thay đổi dữ liệu → dữ liệu đó phải là state ở component cha, và cha truyền xuống hàm để con gọi ngược lên.

2.4 Truyền hàm xuống con

```
// Cha
<JobCard job={job} onDelete={() => handleDelete(job.id)} />

// Con
<button onClick={onDelete}>Xoá</button>             // ✔ truyền tham chiếu hàm
<button onClick={onDelete()}>Xoá</button>           // ✘ GỌI LUÔN khi render!
```

Việc cần làm:

1. Tách trang chủ thành các component nhỏ: Navbar, Hero, HomeCards, JobListings, JobListing
2. Viết `Card` nhận `children` + prop `bg` có giá trị mặc định
3. Truyền dữ liệu job xuống bằng props, thử cả cách liệt kê từng prop và cách `{...job}`
4. Cố tình đặt tên component chữ thường để thấy trang trắng — hiểu vì sao

5. Cố tình viết `onClick={handleDelete()}` để thấy hàm chạy ngay khi render

PART 03

State với useState — tư duy bất biến

Mục tiêu: Hiểu state là gì, vì sao phải cập nhật bất biến, và 3 hiểu lầm phổ biến nhất về `setState`.

3.1 useState cơ bản

```
const [count, setCount] = useState(0);
//        ▲ giá trị hiện tại  ▲ hàm cập nhật    ▲ giá trị khởi tạo (chỉ dùng ở lần render ĐẦU)

setCount(count + 1);           // yêu cầu React render lại với giá trị mới
```

Quy tắc Hook (bắt buộc): chỉ gọi hook ở cấp cao nhất của component — không đặt trong `if`, vòng lặp, hàm lồng. Lý do: React nhận diện hook theo thứ tự gọi; đổi thứ tự giữa các lần render là hỏng toàn bộ state. ESLint plugin `react-hooks` sẽ bắt lỗi này.

3.2 Hiểu lầm 1: "setState cập nhật ngay lập tức"

```
function handleClick() {
  setCount(count + 1);
  console.log(count);       // ✘ vẫn in ra giá trị CŨ
}
```

Vì `count` là hằng số của lần render này (closure — xem Part J). `setCount` chỉ lên lịch một lần render mới; giá trị mới chỉ xuất hiện ở lần render sau. Đây gọi là "state là một snapshot".

3.3 Hiểu lầm 2: gọi setState nhiều lần thì cộng dồn

```
setCount(count + 1);
setCount(count + 1);
setCount(count + 1);           // count = 0 → kết quả là 1, KHÔNG phải 3

// ✔ Dùng dạng hàm cập nhật (updater) khi giá trị mới phụ thuộc giá trị cũ
setCount((prev) => prev + 1);
setCount((prev) => prev + 1);
setCount((prev) => prev + 1);      // → 3 ✔
```

React gộp (batch) các lần `setState` trong cùng một sự kiện thành một lần render — đó là tối ưu, không phải bug. Dạng updater đọc giá trị mới nhất trong hàng đợi nên luôn đúng.

3.4 Hiểu lầm 3: sửa trực tiếp object/array trong state

```
// ✘ SAI — mutate: React so sánh thấy CÙNG tham chiếu → không render lại
jobs.push(newJob);
setJobs(jobs);
form.title = 'X';
setForm(form);

// ✔ ĐÚNG — luôn tạo object/array MỚI
setJobs([...jobs, newJob]);                                    // thêm
setJobs(jobs.filter((j) => j.id !== id));                      // xoá
setJobs(jobs.map((j) => j.id === id ? { ...j, ...patch } : j)); // sửa
setForm({ ...form, title: 'X' });                             // sửa 1 field
```

3.5 Chọn cấu trúc state cho tốt

- **Đừng lưu thứ tính được ra state.** Có `jobs` và `keyword` thì `filteredJobs` chỉ cần tính khi render, không cần state riêng (state thừa = nguồn dữ liệu lệch nhau).
- **Đừng để state trùng lặp.** Lưu `selectedId` thay vì lưu cả object `selectedJob` đã copy.
- **Nhóm state đi cùng nhau.** Form nhiều field → một object, hoặc dùng `useReducer` (Part 08).
- **Đặt state ở đúng độ cao.** Càng gần nơi dùng càng tốt; chỉ nâng lên cha khi có 2 component cùng cần (Part 08).

State vs biến thường: biến thường (`let x = 0`) bị reset sau mỗi lần render và không kích hoạt render. State thì được React giữ lại giữa các lần render và kích hoạt render khi đổi. Cần giữ giá trị mà không muốn render lại (id của timer, tham chiếu DOM) → dùng `useRef`.

Việc cần làm:

1. Viết bộ đếm, chứng minh `setCount(count+1)` ×3 chỉ tăng 1, còn dạng updater tăng 3
2. Log `count` ngay sau `setCount` để thấy giá trị cũ — giải thích bằng "snapshot"
3. Làm ô tìm kiếm lọc danh sách job: không tạo state `filteredJobs`, chỉ tính khi render
4. Viết 3 thao tác thêm / sửa / xoá job trong state theo đúng cách bất biến
5. Cố tình mutate (`jobs.push`) để tận mắt thấy UI không cập nhật
6. Làm form nhiều field bằng một object state + một handler `handleChange` dùng `[e.target.name]`

PART 04

Render danh sách, key & render có điều kiện

Mục tiêu: Render danh sách đúng cách và hiểu thật sự key dùng để làm gì — đây là chỗ gây bug âm thầm nhiều nhất với người mới.

4.1 map để render danh sách

```
<div>
  {jobs.map((job) => (
    <JobListing key={job.id} job={job} />
  ))}
</div>
```

4.2 Key dùng để làm gì

Key không phải để "React biết thứ tự". Key là **danh tính** của phần tử: nó cho React biết "element này ở lần render trước và lần này là cùng một thứ". Nhờ đó React biết nên giữ lại DOM node cùng toàn bộ state bên trong component đó, hay huỷ đi tạo mới.

Cách đặt key	Đánh giá
`key={job.id}`	✔ Đúng — id ổn định từ dữ liệu
`key={index}`	✘ Nguy hiểm khi danh sách có thể sắp xếp / thêm / xoá ở giữa
`key={Math.random()}`	✘ Rất tệ — key đổi mỗi lần render → React huỷ và dựng lại toàn bộ, mất state, mất focus, chậm
Không có key	React cảnh báo trong console và mặc định dùng index

Bug thật khi dùng index làm key: danh sách có ô input; bạn gõ vào ô thứ 2 rồi xoá phần tử thứ 1. Vì key là index, React tưởng phần tử 2 "chính là" phần tử 1 cũ → giữ nguyên state của ô cũ → nội dung bạn gõ nhảy sang dòng khác. Đây là loại bug rất khó truy vết nếu không hiểu key.

4.3 Render có điều kiện

```
{ loading && <Spinner /> }                                    // hiện khi true
{ error ? <Error msg={error} /> : <JobList jobs={jobs} /> }

{ jobs.length > 0
    ? jobs.map((j) => <JobListing key={j.id} job={j} />)
    : <p>Chưa có công việc nào</p> }

// Logic phức tạp → tách ra biến trước khi return, đừng nhồi vào JSX
let content;
if (loading)      content = <Spinner />;
else if (error) content = <Error msg={error} />;
else            content = <JobList jobs={jobs} />;

return <div>{content}</div>;
```

Nhắc lại bẫy từ Part J: `{jobs.length && ...}` in ra số 0 khi mảng rỗng. Luôn viết `{jobs.length > 0 && ...}`. Ngoài ra `null`, `undefined`, `false` đều không render gì — nên `return null` là cách hợp lệ để "không hiện gì".

4.4 Key để chủ động reset state

```
// Đổi key → React huỷ component cũ, tạo mới hoàn toàn (state về khởi tạo)
<JobForm key={selectedJobId} job={selectedJob} />
```

Đây là mẹo chính thống được React khuyến nghị: thay vì viết `useEffect` để "reset form khi đổi job", chỉ cần đổi key.

Việc cần làm:

1. Render danh sách job bằng `map` + key là id
2. Tái hiện bug `key=index`: danh sách có input, xoá phần tử đầu, quan sát giá trị input nhảy sai; sau đó sửa bằng `key=id`
3. Xử lý 3 trạng thái loading / error / empty cho danh sách
4. Tạo bug `{jobs.length && ...}` hiện số 0 rồi sửa lại
5. Dùng mẹo key ở mục 4.4 để reset form khi chọn job khác

PART 05

Sự kiện & Form (controlled component)

> **Mục tiêu:** Xử lý sự kiện đúng cách và làm form theo mô hình controlled — nền tảng cho mọi trang thêm/sửa dữ liệu.

5.1 Sự kiện

```
<button onClick={handleClick}>Lưu</button>                    // truyền tham chiếu
<button onClick={() => handleDelete(job.id)}>Xoá</button>      // cần tham số → bọc arrow

function handleSubmit(e) {
  e.preventDefault();    // chặn browser reload trang khi submit form
}
```

React dùng synthetic event — một lớp bọc chuẩn hoá sự kiện DOM để hoạt động giống nhau trên mọi trình duyệt. API dùng gần như y hệt DOM thật (`e.target` , `e.preventDefault()` , `e.stopPropagation()`).

5.2 Controlled component

```
const [title, setTitle] = useState('');

<input
  value={title}                                  // giá trị LUÔN lấy từ state
  onChange={(e) => setTitle(e.target.value)}     // gõ → cập nhật state → render lại
/>
```

"Controlled" nghĩa là React nắm quyền kiểm soát giá trị của input, DOM chỉ hiển thị lại state. Nhờ vậy bạn validate, format, disable nút submit... đều dễ dàng vì luôn có dữ liệu trong tay.

> **Lỗi thường gặp:** đặt `value={title}` mà quên `onChange` → input bị "đơ", gõ không lên chữ, kèm cảnh báo trong console. Nếu thực sự muốn không kiểm soát thì dùng `defaultValue` .

5.3 Form nhiều field — một handler duy nhất

```
const [form, setForm] = useState({ title: '', type: 'Full-Time', salary: '' });

function handleChange(e) {
  const { name, value, type, checked } = e.target;

  setForm((prev) => ({
    ...prev,
    [name]: type === 'checkbox' ? checked : value,   // computed key
  }));
}

<input name="title"  value={form.title}  onChange={handleChange} />
<input name="salary" value={form.salary} onChange={handleChange} />
```

5.4 Submit

```
async function handleSubmit(e) {
  e.preventDefault();

  if (!form.title.trim()) {
    setError('Tiêu đề là bắt buộc');
    return;
  }

  setSubmitting(true);
  try {
    await createJob(form);
    navigate('/jobs');
  } catch (err) {
    setError(err.message);
  } finally {
    setSubmitting(false);        // finally: chạy cả khi lỗi
  }
}

<button type="submit" disabled={submitting}>
  {submitting ? 'Đang lưu...' : 'Lưu'}
</button>
```

Luôn khoá nút khi đang submit. Không khoá thì user bấm nhiều lần → tạo trùng bản ghi. React 19 có `useActionState` + `useFormStatus` làm việc này gọn hơn — xem Part 11, và mục 11.9 so sánh trực tiếp với cách viết ở đây.

Việc cần làm:

1. Làm trang AddJobPage với form controlled đầy đủ field
2. Dùng một state object + một `handleChange` cho mọi input
3. Validate phía client (bắt buộc nhập title, lương phải là số), hiện lỗi ngay dưới field

4. Khoá nút submit + đổi chữ khi đang gửi
5. Làm trang EditJobPage: đổ dữ liệu cũ vào form rồi cập nhật
6. Bỏ `e.preventDefault()` một lần để thấy trang reload — hiểu vì sao cần nó

PART 06

useEffect — dùng đúng và dùng ít

Mục tiêu: Hiểu Effect thực sự dùng để làm gì (đồng bộ với hệ thống bên ngoài React), và nhận ra phần lớn `useEffect` người mới viết là không cần thiết.

6.1 Cú pháp & 3 dạng dependency

```
useEffect(() => {
  // side effect: gọi API, subscribe, timer, đọc/ghi localStorage...

  return () => {
    // CLEANUP: chạy trước lần effect kế tiếp và khi component unmount
  };
}, [deps]);
```

Dependency	Chạy khi nào
`[]`	Chỉ sau lần render đầu (mount). Dev + StrictMode: chạy 2 lần
`[a, b]`	Sau mount, và mỗi khi a hoặc b đổi tham chiếu/giá trị
Không truyền	Sau MỌI lần render — hầu như luôn là lỗi

6.2 Cleanup — bắt buộc, không phải tuỳ chọn

```
useEffect(() => {
  const id = setInterval(tick, 1000);
  return () => clearInterval(id);       // không có dòng này = rò rỉ timer
}, []);

useEffect(() => {
  window.addEventListener('resize', onResize);
  return () => window.removeEventListener('resize', onResize);
}, []);
```

StrictMode cố tình mount → unmount → mount lại ở dev chính là để ép bạn phát hiện thiếu cleanup. Nếu bật app thấy có 2 timer chạy, đó là effect của bạn chưa dọn dẹp.

6.3 Bẫy dependency: vòng lặp vô hạn

```
// ✘ Object/array tạo mới mỗi lần render → deps LUÔN khác → effect chạy mãi
useEffect(() => { fetchJobs(filters); }, [filters]);  // filters = {} inline ở cha

// ✘ setState không điều kiện trong effect phụ thuộc chính state đó
useEffect(() => { setCount(count + 1); }, [count]);

// ✔ Dùng giá trị primitive làm dependency
useEffect(() => { fetchJobs(type, keyword); }, [type, keyword]);
```

Đừng bao giờ "im lặng" ESLint bằng cách xoá bớt dependency hoặc thêm `// eslint-disable-next-line react-hooks/exhaustive-deps`. Cảnh báo đó gần như luôn đúng — nó đang chỉ ra rằng effect của bạn phụ thuộc thứ mà bạn chưa xử lý. Cách sửa đúng: chuyển hàm vào trong effect, dùng dạng updater của setState, hoặc dùng `useEffectEvent` (mục 6.5).

6.4 Khi nào KHÔNG cần useEffect

Tình huống	Cách đúng
Tính giá trị dẫn xuất từ props/state (lọc, tổng, format)	Tính thẳng khi render — không cần effect + state
Cập nhật state khi props đổi	Tính khi render, hoặc đổi key để reset (Part 04)
Xử lý sau khi user bấm nút	Viết trong event handler, không phải effect
Gửi request khi submit form	Trong `handleSubmit`
Fetch dữ liệu cho trang	Effect được — hoặc tốt hơn: loader của Router (Part 09) / thư viện data fetching (Part 07.4)
Đồng bộ với hệ thống ngoài: timer, websocket, listener, thư viện DOM bên thứ ba	Đây mới là việc của Effect

Câu hỏi tự kiểm tra trước khi viết `useEffect`: "Việc này có phải là đồng bộ với thứ bên ngoài React không?" Nếu không — 90% là bạn không cần Effect.

6.5 useEffectEvent (React 19.2)

```
import { useEffectEvent, useEffect } from 'react';

function ChatRoom({ roomId, theme }) {
  // Tách phần "sự kiện" ra: đọc được theme MỚI NHẤT nhưng KHÔNG làm effect chạy lại
  const onConnected = useEffectEvent(() => {
    showNotification('Đã kết nối!', theme);
  });

  useEffect(() => {
    const conn = createConnection(roomId);
    conn.on('connected', () => onConnected());
    conn.connect();
    return () => conn.disconnect();
  }, [roomId]);      // ✔ KHÔNG đưa onConnected vào deps — đúng theo quy định của hook này
}
```

Đã kiểm chứng theo `react.dev/blog/2025/10/01/react-19-2` và `react.dev/reference/react/useEffectEvent`: `useEffectEvent` là API mới của React 19.2, import trực tiếp từ `'react'`. Trước 19.2, đổi `theme` sẽ khiến effect chạy lại → ngắt và kết nối lại phòng chat một cách vô lý. Effect Event chỉ được gọi bên trong Effect (hoặc Effect Event khác) — không được gọi trong lúc render, không truyền xuống component/hook khác, và **không bao giờ** đưa vào dependency array (nó cố tình có "identity không ổn định" giữa các lần render). Cần `eslint-plugin-react-hooks` bản mới để lint đúng quy tắc này.

Việc cần làm:

1. Viết effect fetch danh sách job khi mount, có cleanup
2. Cố tình bỏ `[]` để thấy vòng lặp fetch vô hạn (mở tab Network), rồi sửa
3. Viết effect có `setInterval`, quan sát ở StrictMode chạy 2 timer khi thiếu cleanup
4. Rà lại toàn bộ code cũ: tìm những `useEffect` chỉ để tính giá trị dẫn xuất và xoá chúng đi
5. Viết effect có dependency là object inline để tái hiện vòng lặp, rồi sửa bằng dependency primitive
6. Thử `useEffectEvent` cho một effect cần đọc state mới nhất mà không muốn chạy lại

PART 07

Gọi API & Custom Hook

> **Mục tiêu:** Fetch dữ liệu đúng chuẩn (loading / error / race condition), rồi đóng gói lại thành custom hook tái sử dụng — và biết khi nào nên đổi sang thư viện.

7.1 Mẫu fetch đầy đủ

```
const [jobs, setJobs] = useState([]);
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);

useEffect(() => {
  const controller = new AbortController();       // để huỷ request cũ

  async function load() {               // khai hàm async BÊN TRONG effect
    try {
      setLoading(true);
      setError(null);

      const res = await fetch(url, { signal: controller.signal });
      if (!res.ok) throw new Error(`HTTP ${res.status}`);

      setJobs(await res.json());
    } catch (err) {
      if (err.name !== 'AbortError') setError(err.message);
    } finally {
      setLoading(false);
    }
  }

  load();
  return () => controller.abort();          // cleanup: huỷ request khi deps
đổi/unmount
}, [url]);
```

Hai lỗi bắt buộc phải tránh: (1) Không viết `useEffect(async () => {...})` — hàm async trả về Promise, còn React mong nhận về hàm cleanup. Phải khai hàm async bên trong rồi gọi. (2) Race condition: user gõ tìm kiếm nhanh → nhiều request bay đi; response về không theo thứ tự → kết quả cũ ghi đè kết quả mới. `AbortController` trong cleanup giải quyết triệt để.

7.2 Tách tầng gọi API ra khỏi component

```
// src/services/jobService.js — component KHÔNG nên biết chi tiết URL/header
const BASE = import.meta.env.VITE_API_URL;

async function request(path, options = {}) {
  const res = await fetch(`${BASE}${path}`, {
    headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' },
    ...options,
  });

  if (!res.ok) throw new Error((await res.json().catch(() => ({}))).message ?? 'Request
failed');
  return res.status === 204 ? null : res.json();
}

export const getJobs    = () => request('/jobs');
export const getJob     = (id) => request(`/jobs/${id}`);
export const createJob  = (data) => request('/jobs', { method: 'POST', body:
JSON.stringify(data) });
export const deleteJob  = (id) => request(`/jobs/${id}`, { method: 'DELETE' });
```

7.3 Custom Hook — tái sử dụng logic có state

```
// src/hooks/useFetch.js — tên PHẢI bắt đầu bằng "use"
export function useFetch(fetcher, deps = []) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    let alive = true;

    setLoading(true);
    fetcher()
      .then((d) => alive && setData(d))
      .catch((e) => alive && setError(e.message))
      .finally(() => alive && setLoading(false));

    return () => { alive = false; };      // chặn setState sau khi unmount
  }, deps);

  return { data, loading, error };
}

// Dùng
const { data: jobs, loading, error } = useFetch(getJobs, []);
```

Custom hook chia sẻ LOGIC, không chia sẻ STATE. Hai component cùng gọi `useFetch` sẽ có hai bộ state riêng biệt. Muốn dùng chung một dữ liệu thì phải nâng state lên cha hoặc dùng Context (Part 08).

7.4 useEffect thủ công hay dùng thư viện (TanStack Query / SWR) — chọn đúng lúc nào

Bạn vừa tự viết `useFetch` ở mục 7.3 — nó chạy tốt cho crash course này. Nhưng dự án thật cần thêm: cache giữa các lần chuyển trang, tự refetch khi quay lại tab, chống gọi trùng request, retry khi lỗi mạng, tự invalidate sau khi tạo/sửa/xoá. Câu hỏi quyết định: **bạn có sẵn sàng tự viết và bảo trì toàn bộ phần cache/retry/dedupe đó bằng tay không?**

Cách 1 — Tự viết bằng useEffect (`useFetch` ở 7.3)

Không thêm dependency, hiểu rõ 100% code đang chạy — đúng mục đích học của crash course này. Đổi lại: mỗi component gọi lại là một lần fetch mới (không cache), tự tay viết lại race condition/cleanup mỗi nơi cần, không tự động refetch.

Cách 2 — TanStack Query (`useQuery` / `useMutation`)

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

function JobsPage() {
  const { data: jobs, isLoading, error } = useQuery({
    queryKey: ['jobs'],
    queryFn: getJobs,          // hàm ở 7.2, không đổi gì cả
  });

  const queryClient = useQueryClient();
  const { mutate: remove } = useMutation({
    mutationFn: deleteJob,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['jobs'] }),
  });
}
```

Cache theo `queryKey`, tự dedupe request trùng, tự refetch khi focus lại tab, và `invalidateQueries` tự fetch lại sau khi mutate thành công — toàn bộ những thứ bạn phải tự viết tay ở Cách 1.

> **Đã kiểm chứng theo `tanstack.com/query/latest` và `swr.vercel.app/docs/getting-started` :** `useQuery({ queryKey, queryFn })` và `useMutation({ mutationFn, onSuccess })` là API hiện tại của `@tanstack/react-query` . SWR (thư viện nhẹ hơn, cùng của đội Vercel) dùng API tối giản hơn: `const { data, error, isLoading } = useSWR(key, fetcher)` — không có khái niệm mutation riêng, thiên về đọc dữ liệu là chính.

Nhu cầu	Cách 1: tự viết	Cách 2: TanStack Query / SWR
Học cơ chế fetch/loading/error/cleanup thật sự	✅ Bắt buộc — đây chính là mục đích	❌ Thư viện che hết chi tiết bên dưới
Cache dữ liệu giữa các trang, tránh gọi lại API thừa	❌ Phải tự viết tầng cache riêng	✅ Có sẵn theo `queryKey`
Tự động refetch khi quay lại tab / mất mạng rồi có lại	❌ Không có sẵn	✅ Bật mặc định

Đồng bộ UI sau khi tạo/sửa/xoá (mutate)	Tự gọi lại fetch thủ công	✅ `invalidateQueries` / SWR `mutate()`
Dự án nhỏ, ít màn hình, không cần chia sẻ cache	✅ Gọn, không thêm dependency	Hơi thừa cho quy mô nhỏ

Quy tắc thực dụng: tự viết ở crash course này trước để hiểu cơ chế (Cách 1) — đó là lý do Part 07 tồn tại. Khi lên dự án thật có nhiều trang cùng cần một dữ liệu, cần cache/retry/refetch, chuyển sang TanStack Query (đầy đủ tính năng hơn) hoặc SWR (nhẹ hơn, học nhanh hơn) thay vì tự mở rộng `useFetch` mãi.

> **Việc cần làm:**
>
> 1. Tạo `src/services/jobService.js`, chuyển toàn bộ lời gọi fetch rải rác trong component về đây
> 2. Viết mẫu fetch đầy đủ ở 7.1 cho trang danh sách (loading / error / dữ liệu rỗng)
> 3. Tái hiện race condition: ô tìm kiếm gõ nhanh, log kết quả nhận về theo thứ tự, rồi sửa bằng `AbortController`
> 4. Đóng gói thành `useFetch` và dùng lại ở ít nhất 2 trang
> 5. Thử tắt json-server để test nhánh lỗi hiển thị đúng
> 6. (Tuỳ chọn, để so sánh) Cài `@tanstack/react-query`, viết lại trang danh sách bằng `useQuery`, tự cảm nhận sự khác biệt so với 7.3

PART 08

Chia sẻ state: lifting up, Context, useReducer

> **Mục tiêu:** Biết chọn đúng công cụ khi nhiều component cần cùng một dữ liệu — và biết khi nào không nên dùng Context.

8.1 Lifting state up (nâng state lên cha)

Hai component cần cùng một dữ liệu → đưa state lên component cha gần nhất của cả hai, rồi truyền xuống bằng props. Đây là cách mặc định và nên là lựa chọn đầu tiên.

```
function JobsPage() {
  const [keyword, setKeyword] = useState('');      // state ở cha

  return (
    <>
      <SearchBar value={keyword} onChange={setKeyword} />
      <JobList keyword={keyword} />
    </>
  );
}
```

8.2 Context — khi props phải xuyên quá nhiều tầng

Vấn đề "props drilling": truyền `user` từ App → Layout → Navbar → UserMenu, trong đó Layout và Navbar không dùng mà chỉ chuyển tiếp. Context giải quyết đúng chuyện đó.

```
// src/context/AuthContext.jsx
import { createContext, useContext, useState } from 'react';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [token, setToken] = useState(() => localStorage.getItem('token'));

  // React 19: dùng thẳng <AuthContext> làm provider, không cần .Provider
  return (
    <AuthContext value={{ user, token, setUser, setToken }}>
      {children}
    </AuthContext>
  );
}

// Hook tiện dụng + báo lỗi rõ ràng nếu dùng ngoài Provider
export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error('useAuth phải dùng bên trong <AuthProvider>');
  return ctx;
}
```

Context không phải state manager và không tối ưu hiệu năng. Khi giá trị context đổi, mọi component đang `useContext` đều render lại. Vì vậy: (1) đừng nhồi mọi thứ vào một context khổng lồ — tách theo mối quan tâm (Auth / Theme / Cart); (2) đừng dùng Context cho dữ liệu đổi liên tục (giá trị input đang gõ); (3) dữ liệu từ server thì nên dùng thư viện data fetching (Part 07.4) thay vì Context.

8.3 useReducer — gom nhiều state liên quan vào một nơi

```
function jobsReducer(state, action) {
  switch (action.type) {
    case 'loading': return { ...state, loading: true, error: null };
    case 'success': return { loading: false, error: null, items: action.payload };
    case 'error':   return { ...state, loading: false, error: action.error };
    case 'remove':  return { ...state, items: state.items.filter((j) => j.id !==
action.id) };
    default:        throw new Error(`Unknown action: ${action.type}`);
  }
}

const [state, dispatch] = useReducer(jobsReducer, { items: [], loading: true, error:
null });

dispatch({ type: 'success', payload: jobs });
```

`useReducer` gom mọi cách state có thể thay đổi vào đúng một hàm thuần (`jobsReducer`) thay vì rải rác nhiều lệnh `setState` khắp component. Component chỉ `dispatch` "đã xảy ra chuyện gì" (`action`), reducer tự quyết định state mới trông ra sao — tách rời "cái gì xảy ra" khỏi "state đổi thế nào".

8.4 useState (+ Context) hay useReducer — chọn đúng lúc nào

Cả hai đều là cách quản lý state trong React — không cái nào "cao cấp hơn" cái nào. Câu hỏi quyết định: **state của bạn có nhiều cách chuyển đổi qua lại lẫn nhau không, hay chỉ là vài giá trị độc lập?**

Cách 1 — nhiều `useState` độc lập (+ Context nếu cần chia sẻ)

```
const [loading, setLoading] = useState(true);
const [error, setError] = useState(null);
const [items, setItems] = useState([]);
// mỗi state độc lập, tự set — dễ đọc khi ít, dễ set SAI thứ tự khi nhiều
```

Đơn giản, không cần định nghĩa action/reducer, dễ đọc khi chỉ có 1-3 giá trị không liên quan chặt tới nhau.

Cách 2 — `useReducer` (như mục 8.3)

Gom cả loading/error/items vào 1 state object, mọi thay đổi đi qua đúng 1 hàm `jobsReducer` — không thể quên set `error: null` khi bắt đầu loading lại, vì logic đó nằm sẵn trong `case 'loading'` .

> **Đã đối chiếu** `react.dev/reference/react/useReducer` : docs chính thức khuyến nghị chuyển từ nhiều `useState` sang `useReducer` đúng khi "một state cập nhật kích hoạt cập nhật của state khác" hoặc số lượng `setState` rải rác trong nhiều handler đã gây khó theo dõi — không có "số lượng state" cụ thể nào là ngưỡng, đây là dấu hiệu định tính.

Nhu cầu	Cách 1: useState	Cách 2: useReducer
1-3 giá trị đơn giản, độc lập với nhau	✅ Gọn, dễ đọc	Rườm rà không cần thiết
Nhiều giá trị liên quan chặt, nhiều nhánh chuyển trạng thái (loading/success/error)	❌ Dễ quên đồng bộ giữa các state	✅ Một hàm reducer kiểm soát toàn bộ nhánh
Cần test riêng logic "state đổi thế nào" mà không cần render component	❌ Logic nằm rải trong handler	✅ reducer là hàm thuần, test độc lập dễ dàng
Chia sẻ cho nhiều component ở xa nhau trong cây	Kết hợp với Context (8.2)	✅ Kết hợp Context + reducer rất hợp (state phức tạp + chia sẻ xa)
Dữ liệu từ server, cần cache/refetch	Không phải việc của cả hai — dùng thư viện data fetching (Part 07.4)	

Quy tắc thực dụng: bắt đầu luôn bằng `useState` . Chỉ chuyển sang `useReducer` khi bạn thấy mình đang set nhiều state cùng lúc trong cùng một handler để giữ chúng "khớp" với nhau (ví dụ: bắt đầu fetch phải vừa `setLoading(true)` vừa `setError(null)`) — đó là dấu hiệu state đã đủ phức tạp để gom vào 1 reducer.

> **Việc cần làm:**
>
> 1. Nâng state ô tìm kiếm lên cha, chia sẻ giữa SearchBar và JobList
> 2. Tạo `AuthContext` + `useAuth` (chưa cần API thật, lưu token giả vào localStorage)
> 3. Bọc `AuthProvider` ở App, đọc user trong Navbar bằng `useAuth` — xoá hết props drilling

4. Cố tình gọi `useAuth` ở component ngoài Provider để thấy lỗi rõ ràng bạn tự throw
5. Chuyển state loading/error/items của trang danh sách sang `useReducer`, so sánh xem code dễ đọc hơn không
6. Tự trả lời câu hỏi ở mục 8.4 cho chính state bạn vừa chuyển: nó có đáng chuyển không?

PART 09

React Router — routing, loader & form action

> **Mục tiêu:** Làm app nhiều trang thật sự: URL đổi, nested layout, lấy tham số, và tải dữ liệu bằng loader thay vì `useEffect`.

> **Về phiên bản (đã kiểm chứng `reactrouter.com`, 09/2026):** từ v7, các package được gộp về tên duy nhất là `react-router` (`react-router-dom` vẫn còn để tương thích ngược, nhưng docs hiện tại import thẳng từ `"react-router"`). Bản mới nhất hiện tại là v8. API 3 mode dưới đây không đổi giữa v7 và v8, nên kiến thức phần này dùng được cho cả hai.

9.1 Ba "mode" của React Router — chọn mode nào cho dự án của bạn

React Router không phải một API duy nhất — nó cung cấp 3 cách dùng khác nhau, đủ khác để coi là 3 "chế độ" riêng biệt. Câu hỏi quyết định: **bạn có cần tải dữ liệu gắn liền với route (loader/action) không, và có cần SSR/type-safe routing không?**

Cách 1 — Declarative mode (`<BrowserRouter>`, `<Routes>`)

Theo đúng nguyên văn docs: "Declarative mode enables basic routing features like matching URLs to components, navigating around the app, and providing active states" — dùng `<Link>`, `useNavigate`, `useLocation`. Không có loader/action.

Cách 2 — Data mode (`createBrowserRouter`, `loader`, `action`)

Theo đúng nguyên văn docs: "Data Mode adds data loading, actions, pending states and more" bằng cách chuyển cấu hình route ra ngoài quá trình render React. Đây là mode cả tài liệu này dùng — dự án job listings của bạn đang ở mode này.

Cách 3 — Framework mode (Vite plugin)

Theo đúng nguyên văn docs: "Framework Mode wraps Data Mode with a Vite plugin" — thêm type-safe `href`, Route Module API, code splitting tự động, và có thể bật SSR/SSG. Cần thiết lập dự án khác (không phải Vite React thuần).

> **Đã đối chiếu `reactrouter.com/start/modes`:** đúng 3 tên "Declarative / Data / Framework", trích nguyên văn mô tả từng mode ở trên. Docs nói rõ "every mode supports any architecture and deployment target" — nghĩa là không có luật cứng "SPA thì bắt buộc mode nào", nhưng với 1 SPA gọi API riêng (đúng bài toán job listings của khoá này), Data mode là lựa chọn thực tế nhất: có loader/action mà không cần đổi hẳn sang kiến trúc framework.

Nhu cầu	Declarative	Data	Framework
App nhỏ, chỉ cần đổi URL, không tải dữ liệu theo route	✅ Đủ dùng, ít khái niệm nhất	Thừa tính năng	Thừa rất nhiều
Tải dữ liệu gắn với route (loader), submit form qua action	❌ Không có	✅ Đúng mục đích	✅ Có (bên trong Data mode)
Type-safe params, code splitting tự động theo route	❌	❌	✅
Cần SSR/SSG	❌	❌	✅ Bắt buộc nếu cần
Dự án Vite SPA gọi API riêng (đúng bài toán khoá học này)	Được nhưng thiếu loader	✅ Vừa đủ	Cần đổi cấu trúc dự án

Quy tắc thực dụng: mới học → bắt đầu Declarative để hiểu `<Link>` / `useNavigate` cơ bản; ngay khi cần tải dữ liệu theo route → chuyển sang Data mode và ở lại đó (đây là mode khoá học dùng suốt Part 09-12). Chỉ cân nhắc Framework mode khi dự án thật sự cần SSR hoặc type-safe routing tự động — đừng chuyển "vì nghe nói mới hơn".

9.2 Cấu hình router (Data mode)

```
import { createBrowserRouter, RouterProvider } from 'react-router';

const router = createBrowserRouter([
  {
    path: '/',
    element: <MainLayout />,                // layout bọc mọi route con
    ErrorBoundary: RootErrorBoundary,       // bắt lỗi loader/action/route không tồn
tại
    children: [
      { index: true, element: <HomePage /> },                    // path '/'
      { path: 'jobs', element: <JobsPage /> },
      { path: 'jobs/:id', element: <JobPage />, loader: jobLoader },
      { path: 'add-job', element: <AddJobPage /> },
      { path: '*', element: <NotFoundPage /> },                      // catch-all
    ],
  },
]);

export default function App() { return <RouterProvider router={router} />; }
```

```
// MainLayout — <Outlet /> là chỗ route con được render vào
import { Outlet } from 'react-router';

export default function MainLayout() {
  return (
    <>
      <Navbar />
      <main><Outlet /></main>
      <Toaster />
    </>
  );
}
```

> **Đã sửa so với bản gốc — đối chiếu `reactrouter.com/start/data/routing` và `reactrouter.com/how-to/error-boundary`:** import Data mode hiện tại lấy thẳng từ `"react-router"` (không phải `"react-router-dom"` — package đó vẫn tồn tại và re-export để không phá code cũ, nhưng docs hiện tại dùng `"react-router"`). Xử lý lỗi cũng đổi tên: docs hiện tại dùng property `ErrorBoundary` (một component, đọc lỗi bằng hook `useRouteError`) thay vì `errorElement` nhận thẳng 1 element — xem chi tiết mục 9.4.

9.3 Điều hướng

```
// ✔ Link: điều hướng client-side, KHÔNG tải lại trang
<Link to={`/jobs/${job.id}`}>Xem chi tiết</Link>

// NavLink: tự biết mình có đang active không → tô sáng menu
<NavLink to="/jobs" className={({ isActive }) => isActive ? 'text-white' : 'text-gray-
400'}>
  Việc làm
</NavLink>

// Điều hướng bằng code (sau khi submit thành công)
const navigate = useNavigate();
navigate('/jobs');
navigate(-1);                              // quay lại như nút Back
navigate('/jobs', { replace: true });      // thay thế history, không back về được

// ✘ <a href="/jobs"> — tải lại toàn bộ app, mất hết state
```

9.4 Lấy tham số & loader

```
// Đọc :id từ URL
const { id } = useParams();

// Đọc query string ?type=Full-Time
const [searchParams, setSearchParams] = useSearchParams();
const type = searchParams.get('type');

// Loader: Router gọi TRƯỚC khi render component → không còn màn hình trắng nhấp nháy
import { data } from 'react-router';

export async function jobLoader({ params, request }) {
  const res = await fetch(`/api/jobs/${params.id}`, { signal: request.signal });

  if (!res.ok) throw data('Not Found', { status: 404 });   // → ErrorBoundary

  return res.json();
}

// Trong component
const job = useLoaderData();
const { state } = useNavigation();         // 'idle' | 'loading' | 'submitting'

// RootErrorBoundary — ErrorBoundary KHÔNG nhận props, đọc lỗi qua hook
import { useRouteError, isRouteErrorResponse } from 'react-router';

function RootErrorBoundary() {
  const error = useRouteError();

  if (isRouteErrorResponse(error)) {
    return <h1>{error.status} {error.statusText}</h1>;   // 404, 500...
  }
  return <h1>Đã có lỗi xảy ra</h1>;
}
```

> **Đã sửa so với bản gốc — đối chiếu `reactrouter.com/how-to/error-boundary`:** bản cũ dùng `throw new Response('Not Found', { status: 404 })` và `errorElement`. Docs hiện tại khuyến nghị helper `data(body, init)` (import từ `'react-router'`) để throw lỗi có status code, kết hợp property `ErrorBoundary` + hook `useRouteError()` + `isRouteErrorResponse()` để phân biệt "lỗi có status code" (404 do bạn chủ động throw) với "lỗi JS bất ngờ" (throw Error thường). Cách cũ (`errorElement` + `throw new Response()`) vẫn chạy được vì thư viện giữ tương thích ngược, nhưng không còn là API được docs trình bày làm chuẩn.

Vì sao loader tốt hơn `useEffect` để fetch: loader chạy song song với việc tải code của trang và trước khi render, nên tránh được "waterfall" (render → effect → fetch → render lại) và không bị hiện tượng nháy loading. Router cũng tự huỷ request khi user điều hướng đi (nhờ `request.signal`) — chính là `AbortController` ở Part 07, nhưng miễn phí.

> **Việc cần làm:**
>
> 1. Dựng router đủ 6 route: home, jobs, job detail, add, edit, 404 — có MainLayout + Outlet

2. Thay mọi `<a href>` nội bộ bằng `Link`; dùng `NavLink` để tô sáng menu đang mở
3. Trang chi tiết: dùng `loader` + `useLoaderData` thay cho `useEffect`
4. Thêm `ErrorBoundary` + `useRouteError`, test bằng cách vào `/jobs/99999`
5. Lọc theo `?type=` bằng `useSearchParams` (bộ lọc lưu trên URL để chia sẻ link được)
6. Dùng `useNavigation()` hiện thanh loading khi đang chuyển trang

PART 10

Hiệu năng: re-render, memo & React Compiler

> **Mục tiêu:** Hiểu vì sao component render lại, và tối ưu đúng chỗ — chứ không rải `useMemo` khắp nơi.

10.1 Khi nào một component render lại

- State của chính nó đổi
- Component cha render lại (mặc định con render theo, dù props không đổi)
- Context nó đang dùng đổi giá trị

Render lại không đồng nghĩa với chậm. React chỉ cập nhật DOM khi có khác biệt thật. Đừng tối ưu trước khi đo — dùng React DevTools Profiler hoặc Performance Tracks (mới có từ React 19.2, tab Performance của Chrome, có track "Scheduler" và "Components") để tìm đúng chỗ nghẽn.

10.2 Ba công cụ memo hoá

API	Ghi nhớ cái gì	Dùng khi
`useMemo(fn, deps)`	Giá trị trả về	Tính toán nặng, hoặc cần object/array giữ nguyên tham chiếu
`useCallback(fn, deps)`	Hàm	Truyền hàm xuống con đã bọc `memo`, hoặc làm dependency của effect
`memo(Component)`	Kết quả render của component	Component nặng, nhận props ít đổi

```
const filtered = useMemo(
  () => jobs.filter((j) => j.title.toLowerCase().includes(keyword.toLowerCase())),
  [jobs, keyword],
);

const handleDelete = useCallback((id) => deleteJob(id), []);

const JobListing = memo(function JobListing({ job, onDelete }) { /* ... */ });
```

Đừng lạm dụng: `memo` chỉ có tác dụng khi props thật sự ổn định (referentially stable). Bọc `memo` mà cha vẫn truyền xuống `{...}` hoặc `() => ...` tạo mới mỗi render thì vô ích — còn tốn thêm chi phí so sánh props. Theo `react.dev/reference/react/memo`: nếu không bọc `useMemo` / `useCallback` ở phía cha, props luôn "mới" và `memo` ở con coi như vô hiệu.

10.3 React Compiler v1.0 — thay đổi cuộc chơi

React Compiler (bản 1.0, phát hành 07/10/2025) là trình biên dịch tự động memo hoá: nó phân tích code lúc build và tự chèn tối ưu tương đương `useMemo` / `useCallback` / `memo` ở những chỗ cần — kể cả những chỗ memo hoá có điều kiện mà bạn khó viết tay chuẩn xác được. Mục tiêu: bạn không phải tự viết memo nữa.

```
npm install --save-dev --save-exact babel-plugin-react-compiler@latest
npm install --save-dev eslint-plugin-react-hooks@latest
```

```
// vite.config.js
react({ babel: { plugins: ['babel-plugin-react-compiler'] } })
```

Đã kiểm chứng `react.dev/blog/2025/10/07/react-compiler-1`: React Compiler v1.0 hoạt động cho cả React và React Native, đã chạy production tại Meta. Dự án tạo mới bằng `create-vite`, `create-next-app`, hoặc Expo SDK 54+ có thể bật compiler ngay từ template, không cần cấu hình tay như trên.

Điều kiện để compiler hoạt động: code phải thuần khiết theo đúng "Rules of React" — không mutate props/state, không side-effect trong thân component. Nói cách khác: viết React cho đúng thì được tối ưu miễn phí. Đây là lý do các quy tắc ở Part 00/03 quan trọng hơn bao giờ hết. `eslint-plugin-react-hooks` bản mới đã tích hợp sẵn các lint rule của chính compiler để cảnh báo trước khi build.

10.4 Chia nhỏ bundle: lazy + Suspense

```
const AddJobPage = lazy(() => import('./pages/AddJobPage'));

<Suspense fallback={<Spinner />}>
  <AddJobPage />
</Suspense>
```

Trang ít dùng (form thêm/sửa, trang admin) không cần nằm trong bundle đầu tiên. Đo bằng `npm run build` — Vite in ra kích thước từng chunk.

10.5 useTransition — giữ UI không đứng

```
const [isPending, startTransition] = useTransition();

function handleSearch(value) {
  setKeyword(value);                          // cập nhật gấp: ô input phải phản hồi ngay
  startTransition(() => setQuery(value));     // cập nhật "không gấp": lọc 5000 dòng
}
```

React sẽ ưu tiên cập nhật gấp trước, và có thể ngắt cập nhật trong transition nếu có việc gấp hơn tới — nhờ đó ô input không bị nghẽn khi danh sách lớn.

> **Việc cần làm:**
>
> 1. Mở React DevTools Profiler, ghi lại một lần gõ tìm kiếm, tìm component render nhiều nhất
> 2. Tạo 5000 job giả, đo độ trễ khi gõ; áp `useMemo` cho hàm lọc rồi đo lại
> 3. Bọc `JobListing` bằng `memo` ; chứng minh nó chỉ có tác dụng khi hàm truyền xuống được `useCallback`
> 4. Dùng `lazy` + `Suspense` cho trang add/edit, so sánh kích thước chunk trước/sau
> 5. Áp `useTransition` cho ô tìm kiếm với danh sách lớn
> 6. (Nâng cao) Bật React Compiler và bỏ bớt memo viết tay, đo lại xem còn nhanh không

---

PART 11

React 19: Actions, hooks mới & use()

> **Mục tiêu:** Nắm những thứ React 19 làm gọn đi so với cách viết cũ — để không học theo tutorial 2022 rồi viết code lỗi thời.

11.1 ref là prop bình thường — không cần forwardRef

```
// React 19 ✔
function Input({ placeholder, ref }) {
  return <input placeholder={placeholder} ref={ref} />;
}

// React 18 và cũ hơn: phải bọc forwardRef
const Input = forwardRef((props, ref) => <input ref={ref} {...props} />);
```

Ngoài ra, callback ref giờ có thể trả về hàm cleanup: `ref={(node) => { ...; return () => {...} }}` — chạy khi component unmount.

11.2 Context dùng trực tiếp làm Provider

```
<ThemeContext value="dark">{children}</ThemeContext>          // React 19
<ThemeContext.Provider value="dark">...</ThemeContext.Provider> // cách cũ, vẫn chạy
```

11.3 Actions: form gọi hàm async trực tiếp

```
// Truyền hàm vào action của <form> — React tự gọi, tự quản lý pending
function AddJobForm() {
  async function createAction(formData) {
    await createJob({
      title: formData.get('title'),
      salary: formData.get('salary'),
    });
  }

  return (
    <form action={createAction}>
      <input name="title" />
      <input name="salary" />
      <SubmitButton />
    </form>
  );
}
```

Form dùng `action` chạy trong một Transition, luôn dùng phương thức POST bất kể prop `method`, và **tự reset các field uncontrolled** sau khi hàm thành công — bạn không cần `onSubmit` + `e.preventDefault()` nữa.

11.4 useActionState — state + pending + error trong 1 hook

```
import { useActionState } from 'react';

const [state, formAction, isPending] = useActionState(
  async (prevState, formData) => {
    try {
      await createJob({ title: formData.get('title') });
      return { ok: true, error: null };
    } catch (e) {
      return { ok: false, error: e.message };    // trả lỗi về state, không cần
useState riêng
    }
  },
  { ok: false, error: null },                     // state khởi tạo
);

<form action={formAction}>
  <input name="title" />
  <button disabled={isPending}>{isPending ? 'Đang lưu...' : 'Lưu'}</button>
  {state.error && <p className="text-red-600">{state.error}</p>}
</form>
```

Tham số thứ 3 (tuỳ chọn) là `permalink` — dùng cho progressive enhancement với Server Functions, crash course này không cần tới. So sánh: cách cũ ở Part 05 cần 3 state (`submitting`, `error`, dữ liệu) + `try/catch/finally` thủ công. Ở đây gói lại trong một hook.

11.5 useFormStatus — nút submit tự biết form đang gửi

```
import { useFormStatus } from 'react-dom';           // chú ý: từ react-dom, không phải
react

function SubmitButton() {
  const { pending } = useFormStatus();          // đọc trạng thái của <form> CHA

  return <button disabled={pending}>{pending ? 'Đang gửi...' : 'Gửi'}</button>;
}
```

Component gọi `useFormStatus` phải được render **bên trong** thẻ `<form>` — gọi ngay trong component render ra chính form đó sẽ không nhận được trạng thái (luôn thấy `pending: false`). Đó là lý do `SubmitButton` phải tách thành component riêng như trên. Nhờ vậy component nút dùng lại được ở mọi form, không cần truyền prop `isPending` xuống.

11.6 useOptimistic — cập nhật lạc quan

```
const [optimisticJobs, addOptimistic] = useOptimistic(
  jobs,
  (current, newJob) => [...current, { ...newJob, sending: true }],
);

async function action(formData) {
  const draft = { title: formData.get('title') };

  addOptimistic(draft);           // hiện NGAY trên UI
  await createJob(draft);          // nếu lỗi → React tự rollback về state thật
}
```

Tham số thứ 2 là reducer tuỳ chọn `(currentState, action) => nextOptimisticState` — ví dụ trên dùng nó để thêm cờ `sending: true`. Khi action thất bại (throw lỗi), `optimisticJobs` tự quay về đúng giá trị thật (`jobs`) vì optimistic state chỉ tồn tại trong lúc action đang pending.

11.7 use() — đọc Promise / Context linh hoạt

```
import { use, Suspense } from 'react';

function JobDetail({ jobPromise }) {
  const job = use(jobPromise);            // "mở" promise; đang chờ → Suspense hiện
fallback
  return <h1>{job.title}</h1>;
}

<Suspense fallback={<Spinner />}><JobDetail jobPromise={p} /></Suspense>
```

Khác biệt so với hook thường: `use()` được gọi trong `if` hoặc sau early return (các hook khác thì không). Nhưng đừng tạo Promise trong thân component (mỗi render tạo mới → loop vô hạn) — promise phải đến từ loader, từ cache, hoặc từ cha.

11.8 Metadata & các tiện ích khác

```
// React 19: đặt <title>, <meta>, <link> ngay trong component — React tự đưa lên <head>
function JobPage({ job }) {
  return (
    <>
      <title>{job.title} | Job Board</title>
      <meta name="description" content={job.description} />
      <h1>{job.title}</h1>
    </>
  );
}
```

API	Tác dụng
`<Activity mode="hidden">`	React 19.2: giữ phần UI ẩn ở trạng thái "tạm dừng" (huỷ effect, hoãn cập nhật) để pre-render trước — hữu ích cho tab, modal
`useEffectEvent`	React 19.2: tách logic sự kiện khỏi dependency của Effect (Part 06.5)
`useId`	Sinh id ổn định cho label/input, an toàn khi SSR
Error boundary tốt hơn	Không còn log lỗi trùng lặp; có `onCaughtError` / `onUncaughtError` ở `createRoot`

11.9 Actions (React 19) hay controlled input + useState (Part 05) — chọn đúng lúc nào

Đây là điểm dễ gây rối nhất cho người mới, vì tutorial cũ trên mạng vẫn dạy đúng cách Part 05 và không hề nhắc tới Actions. Cả hai đều hợp lệ — câu hỏi quyết định: **bạn có cần đọc/validate giá trị input ngay trong lúc gõ (trước khi submit) không?**

Cách 1 — Controlled input + useState + onSubmit (Part 05)

```
const [form, setForm] = useState({ title: '' });
const [submitting, setSubmitting] = useState(false);

<input value={form.title} onChange={(e) => setForm({ ...form, title: e.target.value })}
/>
// → biết giá trị NGAY khi gõ: validate live, disable nút theo độ dài, đếm ký tự còn
lại...
```

Component giữ toàn quyền kiểm soát giá trị mọi lúc — bắt buộc nếu bạn cần validate/format/preview trong lúc gõ (ví dụ: đếm ký tự, tự thêm dấu phẩy vào số tiền, disable nút "Lưu" khi form chưa hợp lệ dù chưa submit).

Cách 2 — Actions (`<form action>` + `useActionState` / `useFormStatus`, mục 11.3-11.5)

```
const [state, formAction, isPending] = useActionState(createAction, { error: null });

<form action={formAction}>
  <input name="title" />   {/* uncontrolled — không có value/onChange */}
  <SubmitButton />
</form>
// → chỉ biết giá trị khi form được submit (FormData), không biết trong lúc gõ
```

Ít code hơn hẳn: không cần state cho từng field, không cần tự quản lý `submitting` / `error`, form tự reset sau khi thành công, nút submit tự biết pending qua `useFormStatus` mà không cần truyền prop.

> **Đã đối chiếu `react.dev/reference/react-dom/components/form` và `react.dev/blog/2024/12/05/react-19`:** hàm trong `action` nhận đúng 1 tham số `FormData` (hoặc `(prevState, formData)` khi dùng cùng `useActionState`), chạy trong 1 Transition, và tự reset các field **uncontrolled** sau khi thành công. Vì action đọc dữ liệu qua `FormData` lúc submit, field bên trong `<form action>` mặc định là uncontrolled (không set `value`) — muốn vừa dùng Actions vừa đọc giá trị realtime thì vẫn phải tự thêm `useState` + `onChange` song song, không có API nào miễn phí cho việc đó.

Nhu cầu	Cách 1: controlled + useState	Cách 2: Actions
Validate/format ngay trong lúc gõ (đếm ký tự, chặn ký tự lạ)	✅ Bắt buộc — có giá trị mọi lúc	❌ Chỉ có giá trị lúc submit
Form đơn giản: vài field, chỉ validate lúc bấm Lưu	Chạy được nhưng nhiều boilerplate hơn	✅ Ít code nhất, tự quản lý pending/reset
Nút submit tự disable khi đang gửi, dùng lại ở nhiều form	Phải tự truyền prop `isPending` xuống	✅ `useFormStatus` đọc thẳng từ form cha
Cần đồng bộ 2 chiều với UI khác ngoài form (ví dụ preview trực tiếp)	✅ State ở component, dễ chia sẻ	❌ Giá trị "giấu" trong FormData tới lúc submit
Dự án cũ, tutorial cũ đã lỡ viết theo Part 05	Không sai — vẫn là API hợp lệ, không bị xoá	Cân nhắc migrate dần cho form mới, không bắt buộc đổi hết code cũ

Quy tắc thực dụng: form mới, không cần validate realtime phức tạp → viết bằng Actions ngay từ đầu (ít code, đúng hướng React 19). Chỉ quay lại controlled + `useState` khi bạn thực sự cần biết giá trị input trước khi submit (đếm ký tự, preview, validate từng phím gõ) — hai cách có thể trộn trong cùng 1 form nếu cần.

> **Việc cần làm:**
>
> 1. Viết lại form thêm job bằng `<form action>` + `useActionState`, so sánh số dòng với bản Part 05
> 2. Tách `SubmitButton` dùng `useFormStatus`, dùng lại ở 2 form khác nhau
> 3. Áp `useOptimistic` cho hành động xoá job: item biến mất ngay, lỗi thì hiện lại
> 4. Đặt `<title>` động cho trang chi tiết, kiểm tra tab trình duyệt đổi tên
> 5. Nếu code cũ còn `forwardRef` hoặc `.Provider` — viết lại theo cú pháp React 19
> 6. Tự quyết định theo mục 11.9: form tìm kiếm (cần lọc realtime) nên giữ controlled, form thêm job (chỉ cần submit) nên chuyển sang Actions — làm cả hai để thấy khác biệt

PART 12 — FULLSTACK

Kết nối API thật (Laravel / NestJS) & build production

> **Mục tiêu:** Bỏ json-server, nối React vào API bạn tự viết. Đây là phần biến 3 crash course rời rạc thành một hệ thống fullstack.

12.1 Bản đồ hệ thống

```
Browser          React (Vite dev: localhost:5173 · production: file tĩnh trong dist/)
    |
    ▼ HTTP + JSON
API              Laravel (localhost:8000/api/v1) hoặc NestJS (localhost:3000)
    |
    ▼
Database         MySQL / PostgreSQL / SQLite
```

Hai domain khác nhau (5173 ↔ 8000) = cross-origin → phải cấu hình CORS ở phía API, nếu không trình duyệt sẽ chặn response.

12.2 CORS — lỗi đầu tiên ai cũng gặp

```
// Thông báo quen thuộc trong console:
Access to fetch at 'http://localhost:8000/api/v1/jobs' from origin
'http://localhost:5173' has been blocked by CORS policy
```

Phía	Cách xử lý
Laravel	`php artisan config:publish cors` → khai `allowed_origins => ['http://localhost:5173']`
NestJS	`app.enableCors({ origin: 'http://localhost:5173' })` trong `main.ts`
React (dev, cách khác)	Dùng proxy của Vite — request cùng origin nên không cần CORS khi dev

```
// vite.config.js — proxy khi dev (mẹo rất tiện)
server: {
  proxy: { '/api': { target: 'http://localhost:8000', changeOrigin: true } },
}
// Từ React chỉ cần fetch('/api/v1/jobs') — Vite chuyển tiếp sang backend
```

> Đừng để `allowed_origins: ['*']` ở production. Và nhớ: CORS là cơ chế của trình duyệt, không phải bảo mật server — nó không thay thế cho việc xác thực token.

12.3 Xác thực: lưu token ở đâu

Nơi lưu	Ưu	Nhược
localStorage	Đơn giản, sống qua refresh	Đọc được bằng JS → bị đánh cắp nếu có lỗ hổng XSS
Memory (state/context)	An toàn nhất với XSS	Mất khi refresh trang
Cookie HttpOnly	JS không đọc được → chống XSS	Cần backend set cookie + lo CSRF; phải cùng site

Với crash course: dùng localStorage + AuthContext (Part 08) cho đơn giản, nhưng phải biết đây là đánh đổi. Dự án thật nên dùng cookie HttpOnly do backend set (Sanctum SPA mode của Laravel làm đúng việc này).

```
// Gắn token vào mọi request — sửa hàm request() ở Part 07
const token = localStorage.getItem('token');

const res = await fetch(url, {
  headers: {
    'Content-Type': 'application/json',
    'Accept': 'application/json',                    // BẮT BUỘC với Laravel để nhận lỗi JSON
    ...(token && { Authorization: `Bearer ${token}` }),
  },
});

if (res.status === 401) {                            // token hết hạn
  localStorage.removeItem('token');
  window.location.href = '/login';
}
```

12.4 Map lỗi API sang UI

Status	Ý nghĩa	UI nên làm gì
422	Validation sai, có `errors` theo từng field	Hiện lỗi ngay dưới đúng field đó
401	Chưa/không còn đăng nhập	Xoá token, chuyển tới trang login
403	Không có quyền	Thông báo "bạn không có quyền", không chuyển login
404	Không tồn tại	Trang "không tìm thấy"
429	Quá nhiều request	Báo thử lại sau, đọc header `Retry-After`
500	Lỗi server	Thông báo chung + nút thử lại. Không hiện chi tiết lỗi cho user

```
// Nhận lỗi 422 của Laravel và đổ vào form
if (res.status === 422) {
  const { message, errors } = await res.json();
  setFieldErrors(errors);         // { title: ['...'], salary: ['...'] }
  return;
}
```

12.5 Build & deploy

```
npm run build          # → dist/ : index.html + assets có hash
npm run preview        # chạy thử bản build ở local
```

- **SPA fallback:** server phải trả `index.html` cho mọi đường dẫn, nếu không F5 tại `/jobs/5` sẽ ra 404. Nginx: `try_files $uri /index.html;`
- Biến môi trường được nhúng lúc build — đổi `VITE_API_URL` thì phải build lại.
- Đặt React và API sau cùng một domain (vd `app.example.com` + `app.example.com/api`) để khỏi phải lo CORS và dùng được cookie HttpOnly.

Việc cần làm:

1. Chạy song song: API Laravel (hoặc NestJS) ở cổng 8000 và React ở 5173
2. Sửa `jobService.js` trỏ sang API thật qua `VITE_API_URL`
3. Gặp lỗi CORS → sửa ở phía API; sau đó thử lại bằng cách dùng proxy của Vite, hiểu khác biệt hai cách
4. Làm trang login gọi API thật, lưu token, gắn Bearer vào mọi request tiếp theo
5. Xử lý 401 (tự logout), 403, 422 (đổ lỗi vào field), 500 (thông báo chung)
6. Bảo vệ route: chưa đăng nhập thì `/add-job` tự chuyển sang `/login`
7. Build production, chạy preview, F5 tại URL con để kiểm tra SPA fallback

---

PART 13 — TUỲ CHỌN

Testing với Vitest + React Testing Library

Mục tiêu: Test component theo cách người dùng thật tương tác, chứ không test chi tiết cài đặt bên trong.

13.1 Cài đặt

```
npm i -D vitest @testing-library/react @testing-library/user-event \
  @testing-library/jest-dom jsdom
```

```
// vite.config.js
test: { environment: 'jsdom', globals: true, setupFiles: './src/setupTests.js' }
```

```
// src/setupTests.js
import '@testing-library/jest-dom';
```

13.2 Triết lý: test như người dùng

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';

test('hiện tiêu đề job', () => {
  render(<JobListing job={{ id: 1, title: 'React Dev' }} />);
  expect(screen.getByText('React Dev')).toBeInTheDocument();
});

test('gõ tìm kiếm thì lọc danh sách', async () => {
  const user = userEvent.setup();
  render(<JobsPage />);

  await user.type(screen.getByRole('textbox', { name: /tìm/i }), 'react');

  expect(await screen.findByText('React Dev')).toBeInTheDocument();
  expect(screen.queryByText('PHP Dev')).not.toBeInTheDocument();
});
```

Query	Khi nào dùng
`getByRole`	Ưu tiên số 1 — giống cách người dùng & screen reader tìm phần tử
`getByLabelText`	Input trong form (đồng thời kiểm tra accessibility)
`getByText`	Nội dung hiển thị
`findBy*`	Phần tử xuất hiện sau khi async xong (tự chờ)
`queryBy*`	Khi cần khẳng định không tồn tại (không throw)
`getByTestId`	Phương án cuối — dùng nhiều nghĩa là markup chưa accessible

Đừng test cái này: state nội bộ, tên hàm, số lần render. Hãy test hành vi: "gõ vào ô này thì thấy kết quả kia", "bấm xoá thì item biến mất". Nhờ vậy bạn refactor bên trong mà test vẫn xanh.

13.3 Mock API

```
// Cách nhanh: mock fetch
global.fetch = vi.fn(() =>
  Promise.resolve({ ok: true, json: () => Promise.resolve([{ id: 1, title: 'React Dev'
}]) }));

// Cách chuẩn hơn cho dự án thật: MSW (Mock Service Worker) — giả lập ở tầng network
```

Việc cần làm (tuỳ chọn):

1. Cấu hình Vitest + jsdom, chạy được `npm run test`
2. Test `JobListing` render đúng title/type/salary
3. Test form thêm job: bỏ trống title → hiện lỗi validate
4. Test danh sách với fetch được mock: loading → có dữ liệu
5. Test nhánh lỗi: fetch reject → hiện thông báo lỗi

PART 14 — ĐI LÀM

TypeScript cho React

Mục tiêu: Đọc và viết được component có kiểu: props, state, sự kiện, generic. Phần lớn công ty hiện nay dùng React với TypeScript.

Nên học chương JS (`js-crash-course`) trước. Part 14–20 kế thừa dự án job listings, nhưng code tại đây là phiên bản viết bằng TypeScript. Máy chỉ cần Node 20 trở lên.

14.1 Thêm TypeScript vào dự án Vite

```
npm i -D typescript @types/react @types/react-dom
# Đổi đuôi file có JSX từ .jsx sang .tsx, file không có JSX từ .js sang .ts
npx tsc --noEmit        # kiểm tra kiểu, không sinh file
```

File `tsconfig.json` tối thiểu đã chạy được:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["ES2023", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "moduleResolution": "bundler",
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "verbatimModuleSyntax": true,
    "skipLibCheck": true,
    "noEmit": true
  },
  "include": ["src"]
}
```

Dự án mới thì tạo bằng mẫu `react-ts` của Vite. Bật `strict` ngay từ đầu, vì bật về sau rất tốn công sửa.

> **Vite không kiểm tra kiểu.** Vite chỉ xoá kiểu rồi chạy, nên code sai kiểu vẫn hiện lên trình duyệt. Phải có bước `tsc --noEmit` trong lệnh build và trong CI.

14.2 Mô tả dữ liệu từ API

```
export type JobType = 'full-time' | 'part-time' | 'remote';

export interface Company {
  name: string;
  city?: string; // có thể không có
}

export interface Job {
  id: number;
  title: string;
  type: JobType;
  salary: number;
  company: Company;
}

// Dữ liệu gửi lên khi tạo mới: server tự sinh id
export type NewJob = Omit<Job, 'id'>;

// Kiểu dùng chung cho mọi API phân trang
export interface Paginated<T> {
  data: T[];
  meta: { page: number; perPage: number; total: number; lastPage: number };
}
```

`type` và `interface` đều dùng được cho object. Dùng `type` khi cần union như `JobType`. Kiểu chỉ tồn tại lúc biên dịch, nên dữ liệu thật từ API vẫn có thể sai. Muốn kiểm tra lúc chạy phải dùng Zod (Part 16).

14.3 Props và children

```
interface JobCardProps {
  job: Job;
  highlight?: boolean; // prop không bắt buộc
  onSelect: (id: number) => void; // hàm cha truyền xuống
  children?: ReactNode; // nội dung lồng bên trong
}

export function JobCard({ job, highlight = false, onSelect, children }: JobCardProps) {
  return (
    <article className={highlight ? 'card card--hot' : 'card'}>
      <h3>{job.title}</h3>
      <p>{job.company.city ?? 'Chưa rõ địa điểm'}</p>
      <button onClick={() => onSelect(job.id)}>Xem</button>
      {children}
    </article>
  );
}
```

14.4 State và sự kiện

```
export function SearchBox({ onSearch }: { onSearch: (q: string) => void }) {
  const [q, setQ] = useState(''); // tự suy ra kiểu string
  const [selected, setSelected] = useState<Job | null>(null); // phải khai báo: ban đầu
là null

  const handleChange = (e: ChangeEvent<HTMLInputElement>) => setQ(e.target.value);

  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
        onSearch(q);
      }}
    >
      <label htmlFor="q">Tìm kiếm</label>
      <input id="q" value={q} onChange={handleChange} />
      {selected && <p>{selected.title}</p>}
      <button onClick={() => setSelected(null)} type="submit">Tìm</button>
    </form>
  );
}
```

Cần mô tả	Viết
State giá trị đơn giản	`useState('')`, TypeScript tự suy ra
State ban đầu là `null` hoặc mảng rỗng	`useState<Job | null>(null)`, `useState<Job[]>([])`
Sự kiện input	`ChangeEvent<HTMLInputElement>`

Cần mô tả	Viết
Sự kiện form	`FormEvent<HTMLFormElement>`
Sự kiện click	`MouseEvent<HTMLButtonElement>`
Ref tới phần tử	`useRef<HTMLInputElement>(null)`
Nội dung lồng bên trong	`ReactNode`

Mẹo: viết hàm ngay trong JSX thì TypeScript tự biết kiểu của `e`. Chỉ cần ghi kiểu khi tách hàm ra ngoài.

14.5 Component generic

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => ReactNode;
  getKey: (item: T) => string | number;
}

export function List<T>({ items, renderItem, getKey }: ListProps<T>) {
  return <ul>{items.map((item) => <li key={getKey(item)}>{renderItem(item)}</li>)}
</ul>;
}
```

Component này dùng được cho danh sách bất kỳ, và `renderItem` biết chính xác kiểu của `item`.

14.6 Lỗi hay gặp

Lỗi	Nguyên nhân	Sửa
`Object is possibly 'null'`	Giá trị có thể `null`	`?.`, kiểm tra trước, hoặc `??`
`Property 'x' does not exist on type 'never'`	`useState([])` mà không ghi kiểu	`useState<Job[]>([])`
`Type 'string' is not assignable to type 'JobType'`	Mảng hoặc object bị suy rộng thành `string`	Ghi kiểu rõ ràng hoặc dùng `as const`
Dùng `any` cho nhanh	Tắt mất kiểm tra	Dùng `unknown` rồi thu hẹp kiểu
`as Job` dữ liệu từ API	`as` chỉ là lời hứa, không kiểm tra thật	Chấp nhận với API đáng tin, còn lại dùng Zod

Việc cần làm:

1. Chuyển dự án job listings sang TypeScript, bật `strict`, `npx tsc --noEmit` không còn lỗi.
2. Viết `types/job.ts` khớp với `jobs.json`, không dùng `any`.
3. Gõ kiểu cho toàn bộ props của `JobListing`, `Hero`, `Card`, và form thêm job.
4. Viết component generic `List<T>` và dùng cho danh sách job.
5. Thêm `tsc --noEmit` vào script `build` trong `package.json`.

PART 15 — ĐI LÀM

TanStack Query thực chiến: cache, phân trang, mutation

> **Mục tiêu:** Bỏ cách tự viết `useEffect` + `loading` + `error` cho từng API. Part 7.4 mới so sánh, Part này mới làm thật.

15.1 Thư viện này giải quyết gì

Dữ liệu từ API là **server state**: nó có thể cũ, được nhiều màn hình dùng chung, và cần tải lại. TanStack Query lo cache, gộp request trùng, tải lại, huỷ request cũ, và trạng thái loading/error.

```
npm i @tanstack/react-query
```

```
const queryClient = new QueryClient({
  defaultOptions: { queries: { retry: 1, refetchOnWindowFocus: false } },
});

export function App() {
  return (
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <AuthProvider>
          <Suspense fallback={<p>Đang tải trang...</p>}>
            <Routes>
              <Route path="/login" element={<LoginPage />} />
              <Route element={<ProtectedRoute />}>
                <Route path="/" element={<JobsPage />} />
                <Route path="/jobs/new" element={<JobForm />} />
              </Route>
            </Routes>
          </Suspense>
        </AuthProvider>
      </BrowserRouter>
    </QueryClientProvider>
  );
}
```

15.2 Query key

```
// Gom key một chỗ để không gõ sai và dễ invalidate đúng phạm vi
export const jobKeys = {
  all: ['jobs'] as const,
  list: (filters: JobFilters) => ['jobs', 'list', filters] as const,
};
```

Key là địa chỉ của cache. Cùng key thì dùng chung dữ liệu và chỉ gọi API một lần. Mọi biến ảnh hưởng đến kết quả (trang, từ khoá) đều phải nằm trong key.

15.3 Tầng gọi API và hook đọc dữ liệu

```
export interface JobFilters {
  page: number;
  q: string;
}

export const jobsApi = {
  list: async ({ page, q }: JobFilters, signal?: AbortSignal) => {
    const res = await http.get<Paginated<Job>>('/jobs', { params: { page, q }, signal
});
    return res.data;
  },
  create: async (job: NewJob) => (await http.post<Job>('/jobs', job)).data,
  remove: async (id: number) => {
    await http.delete(`/jobs/${id}`);
  },
};
```

```
export function useJobs(filters: JobFilters) {
  return useQuery({
    queryKey: jobKeys.list(filters), // filters đổi là tự gọi lại
    queryFn: ({ signal }) => jobsApi.list(filters, signal), // signal để huỷ request cũ
    placeholderData: keepPreviousData, // giữ trang cũ trong lúc tải trang mới
    staleTime: 30_000, // 30 giây đầu coi là còn mới, không gọi lại
  });
}
```

Khái niệm	Ý nghĩa
`isPending`	Chưa có dữ liệu lần nào (hiện khung tải)
`isFetching`	Đang gọi API, có thể đã có dữ liệu cũ
`staleTime`	Bao lâu thì dữ liệu bị coi là cũ. Mặc định `0`, nghĩa là cũ ngay
`gcTime`	Bao lâu sau khi không còn component dùng thì xoá khỏi cache. Mặc định 5 phút
`placeholderData: keepPreviousData`	Giữ trang cũ khi đang tải trang mới, tránh màn hình nhảy

Từ bản 5: `isLoading` cũ đổi thành `isPending`, và `keepPreviousData: true` đổi thành `placeholderData: keepPreviousData`. Nếu đọc bài viết cũ thấy khác thì đây là lý do.

15.4 Phân trang lưu trên URL

```
export function JobsPage() {
  const [params, setParams] = useSearchParams(); // state nằm trên URL: F5 hay gửi link
vẫn đúng trang
  const page = Number(params.get('page') ?? 1);
  const q = params.get('q') ?? '';
  const filters = { page, q };

  const { data, isPending, isError, error, isPlaceholderData } = useJobs(filters);
  const deleteJob = useDeleteJob(filters);

  const goTo = (next: number) =>
    setParams((prev) => {
      prev.set('page', String(next));
      return prev;
    });

  if (isPending) return <p>Đang tải...</p>;
  if (isError) return <p role="alert">{getErrorMessage(error)}</p>;

  return (
    <section style={{ opacity: isPlaceholderData ? 0.5 : 1 }}>
      {data.data.length === 0 && <p>Chưa có tin nào</p>}
      <ul>
        {data.data.map((job) => (
          <li key={job.id}>
            {job.title}
            <button onClick={() => deleteJob.mutate(job.id)}>Xoá {job.title}</button>
          </li>
        ))}
      </ul>
      <button disabled={page <= 1} onClick={() => goTo(page - 1)}>Trước</button>
      <span>Trang {data.meta.page}/{data.meta.lastPage}</span>
      <button disabled={isPlaceholderData || page >= data.meta.lastPage} onClick={() =>
goTo(page + 1)}>Sau</button>
    </section>
  );
}
```

Lưu trang và từ khoá trên URL thay vì `useState`. Nhờ vậy F5 vẫn đúng trang, gửi link cho người khác vẫn đúng kết quả, và nút Back hoạt động. Nút "Sau" bị khoá khi `isPlaceholderData` là `true`, để người dùng không bấm vượt quá trang có thật.

15.5 Ghi dữ liệu với mutation

```
export function useCreateJob() {
  const queryClient = useQueryClient();
  return useMutation({
    mutationFn: jobsApi.create,
    onSuccess: () => queryClient.invalidateQueries({ queryKey: jobKeys.all }),
  });
}
```

Sau khi ghi, `invalidateQueries` đánh dấu mọi query có key bắt đầu bằng `['jobs']` là cũ và tải lại các query đang hiển thị. Đây là lý do nên gom key theo cấp.

15.6 Optimistic update

Giao diện đổi ngay, không chờ server. Nếu lỗi thì phục hồi lại.

```
export function useDeleteJob(filters: JobFilters) {
  const queryClient = useQueryClient();
  const key = jobKeys.list(filters);

  return useMutation({
    mutationFn: jobsApi.remove,
    onMutate: async (id: number) => {
      await queryClient.cancelQueries({ queryKey: key }); // tránh request cũ ghi đè
      const previous = queryClient.getQueryData<Paginated<Job>>(key);
      queryClient.setQueryData<Paginated<Job>>(key, (old) =>
        old && { ...old, data: old.data.filter((j) => j.id !== id) },
      );
      return { previous }; // đưa sang onError để phục hồi
    },
    onError: (_error, _id, context) => {
      if (context?.previous) queryClient.setQueryData(key, context.previous);
    },
    onSettled: () => queryClient.invalidateQueries({ queryKey: jobKeys.all }),
  });
}
```

Chỉ dùng cho thao tác hay thành công và dễ phục hồi, như xoá hay thích. Không dùng cho thanh toán.

> **Đã kiểm chứng bằng test:** tải dữ liệu rồi chuyển trang theo URL, hiện lỗi khi API trả 500, và xoá optimistic: dòng biến mất ngay, server trả lỗi thì dòng hiện lại.

> **Việc cần làm:**
>
> 1. Thay toàn bộ `useEffect` gọi API của dự án bằng `useQuery`.
> 2. Gom key vào một object như `jobKeys`.
> 3. Thêm phân trang và ô tìm kiếm lưu trên URL, kết hợp debounce (bài 4.5 trong tài liệu JS) để không gọi API mỗi lần gõ.
> 4. Thêm job bằng `useMutation`, chứng minh danh sách tự cập nhật.

5. Xoá job bằng optimistic update. Tắt API rồi bấm xoá, xác nhận dòng hiện lại và có thông báo lỗi.
6. Mở Network: cùng một trang vào hai lần trong 30 giây thì chỉ có một request.

PART 16 — ĐI LÀM

Form với React Hook Form + Zod

Mục tiêu: Làm form có validate rõ ràng, hiển thị được lỗi từ server, dùng được với trình đọc màn hình.

16.1 Vì sao không tự viết

Form thật có rất nhiều trạng thái: giá trị, lỗi, đã chạm, đang gửi, lỗi server. React Hook Form lo phần đó và hạn chế render lại. Zod mô tả quy tắc ở một chỗ, và **TypeScript tự suy ra kiểu từ quy tắc**.

```
npm i react-hook-form zod @hookform/resolvers
```

16.2 Schema

```
export const jobFormSchema = z.object({
  title: z.string().trim().min(3, 'Tiêu đề tối thiểu 3 ký tự'),
  type: z.enum(['full-time', 'part-time', 'remote'], { error: 'Chọn loại công việc' }),
  salary: z.coerce.number({ error: 'Lương phải là số' }).int('Lương phải là số
nguyên').positive('Lương phải lớn hơn 0'),
  companyName: z.string().trim().min(1, 'Nhập tên công ty'),
});

// input: dữ liệu trong form (salary còn là chuỗi); output: sau khi Zod xử lý (salary
là number)
export type JobFormInput = z.input<typeof jobFormSchema>;
export type JobFormValues = z.output<typeof jobFormSchema>;
```

Ô input luôn trả về chuỗi, nên `salary` cần `z.coerce.number()`. Vì đầu vào và đầu ra khác kiểu, phải tách `z.input` và `z.output`. Lưu ý: chuỗi rỗng được ép thành số `0`, nên phải có `.positive()` thì mới bắt được ô bỏ trống.

16.3 Form

```
export function JobForm({ onCreated }: { onCreated?: () => void }) {
  const createJob = useCreateJob();
  const {
    register,
    handleSubmit,
    setError,
    reset,
    formState: { errors, isSubmitting },
  } = useForm<JobFormInput, unknown, JobFormValues>({
    resolver: zodResolver(jobFormSchema),
    defaultValues: { title: '', salary: '', companyName: '' },
  });

  const onSubmit = async (values: JobFormValues) => {
    try {
      await createJob.mutateAsync({
        title: values.title,
        type: values.type,
        salary: values.salary,
        company: { name: values.companyName },
      });
      reset();
      onCreated?.();
    } catch (error) {
      const fields = getFieldErrors(error); // lỗi 422 từ server gắn vào đúng ô
      for (const [field, message] of Object.entries(fields)) {
        setError(field as keyof JobFormInput, { message });
      }
      if (Object.keys(fields).length === 0) setError('root', { message:
getErrorMessage(error) });
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} noValidate>
      <label htmlFor="title">Tiêu đề</label>
      <input
        id="title"
        aria-invalid={!!errors.title}
        aria-describedby={errors.title ? 'title-error' : undefined}
        {...register('title')}
      />
      {errors.title && <p id="title-error" role="alert">{errors.title.message}</p>}

      <label htmlFor="type">Loại</label>
      <select id="type" defaultValue="" {...register('type')}>
        <option value="" disabled>Chọn...</option>
        <option value="full-time">Toàn thời gian</option>
        <option value="remote">Từ xa</option>
      </select>
      {errors.type && <p role="alert">{errors.type.message}</p>}
```

```
      <label htmlFor="salary">Lương</label>
      <input id="salary" inputMode="numeric" {...register('salary')} />
      {errors.salary && <p role="alert">{errors.salary.message}</p>}

      <label htmlFor="companyName">Công ty</label>
      <input id="companyName" {...register('companyName')} />
      {errors.companyName && <p role="alert">{errors.companyName.message}</p>}

      {errors.root && <p role="alert">{errors.root.message}</p>}
      <button disabled={isSubmitting}>{isSubmitting ? 'Đang lưu...' : 'Lưu'}</button>
    </form>
  );
}
```

16.4 Đưa lỗi từ server vào đúng ô

Validate ở client chỉ để trải nghiệm tốt hơn. **Server vẫn phải validate**, và có lỗi chỉ server mới biết, như email đã tồn tại.

```
// Laravel trả 422 { message, errors: { title: ['...'] } }
// NestJS (ValidationPipe) trả 400 { message: ['title must be...'] }
export function getFieldErrors(error: unknown): Record<string, string> {
  if (!axios.isAxiosError(error)) return {};
  const errors = error.response?.data?.errors as Record<string, string[]> | undefined;
  if (!errors) return {};
  return Object.fromEntries(Object.entries(errors).map(([field, list]) => [field,
list[0]]));
}

export function getErrorMessage(error: unknown): string {
  if (axios.isAxiosError(error)) {
    if (!error.response) return 'Không kết nối được máy chủ';
    const message = error.response.data?.message;
    return Array.isArray(message) ? message.join(', ') : (message ?? `Lỗi
${error.response.status}`);
  }
  return 'Có lỗi xảy ra';
}
```

16.5 Accessibility cho form

- Mọi ô có `label` gắn đúng `htmlFor`, không dùng `placeholder` thay nhãn.
- Ô lỗi có `aria-invalid`, nối với thông báo bằng `aria-describedby`.
- Thông báo lỗi có `role="alert"` để trình đọc màn hình đọc lên.
- Test bằng `getByLabelText` (Part 13) vừa kiểm tra chức năng vừa kiểm tra nhãn.

16.6 Khi nào dùng Actions của React 19

	React Hook Form + Zod	`useActionState` (Part 11)
Validate khi đang gõ	Có sẵn	Tự viết
Form lớn, nhiều trường phụ thuộc nhau	Rất hợp	Khó
Thêm thư viện	Có	Không
Hợp với	SPA gọi API	Form đơn giản, Server Actions của Next.js

Đã kiểm chứng bằng test: Zod báo đủ lỗi và không gọi API; dữ liệu gửi đi đúng kiểu (`salary` là số, `title` đã cắt khoảng trắng); lỗi 422 của server hiện đúng dưới ô. Chạy với zod 4.6, react-hook-form 7.91.

Việc cần làm:

1. Viết lại form thêm và sửa job bằng React Hook Form + Zod, dùng chung một schema.
2. Khi sửa, nạp dữ liệu cũ vào form bằng `values` hoặc `reset`.
3. Hiện lỗi 422 của Laravel vào đúng ô bằng `setError`.
4. Khoá nút khi đang gửi, chặn gửi hai lần.
5. Test: bỏ trống hiện lỗi, nhập đúng thì gọi API đúng dữ liệu, server trả 422 thì hiện lỗi đúng chỗ.

PART 17 — ĐI LÀM

Đăng nhập thực chiến: axios, refresh token, protected route

Mục tiêu: Làm luồng đăng nhập dùng được trong dự án thật: token an toàn, tự làm mới, F5 không mất phiên. Backend tương ứng ở NestJS Part 27.

17.1 Mô hình

	Lưu ở đâu	Sống bao lâu	Vì sao
Access token	Biến trong bộ nhớ	15 phút	XSS khó lấy hơn `localStorage`, lộ cũng hết hạn nhanh
Refresh token	Cookie `HttpOnly`, do server đặt	30 ngày	JavaScript không đọc được

F5 làm mất access token, nên khi tải trang, app gọi `/auth/refresh` để lấy lại.

17.2 Axios instance

```
export const http = axios.create({ baseURL: '/api', withCredentials: true });

let accessToken: string | null = null; // chỉ giữ trong bộ nhớ, không để localStorage
let onSessionExpired: () => void = () => {};

export const setAccessToken = (token: string | null) => {
  accessToken = token;
};
export const setSessionExpiredHandler = (fn: () => void) => {
  onSessionExpired = fn;
};

http.interceptors.request.use((config) => {
  if (accessToken) config.headers.Authorization = `Bearer ${accessToken}`;
  return config;
});

// API NestJS bọc mọi response thành { success, data } (TransformInterceptor, Part 07
của plan NestJS).
// Bóc ở MỘT chỗ để phần còn lại của app chỉ thấy dữ liệu. Có meta thì giữ nguyên để
lấy phân trang.
function unwrap<T extends AxiosResponse>(res: T): T {
  if (res.data?.success === true && !('meta' in res.data)) res.data = res.data.data;
  return res;
}
```

Đoạn `unwrap` tồn tại vì API NestJS của bạn bọc response trong `{ success, data }`. Nếu quên, đăng nhập sẽ không chạy, vì `res.data.access_token` là `undefined`. **Lỗi này không test giả lập nào bắt được, chỉ lộ ra khi nối với API thật.**

17.3 Tự làm mới token

```
type RetriableConfig = InternalAxiosRequestConfig & { _retried?: boolean };

// Nhiều request cùng nhận 401 chỉ gọi refresh MỘT lần và cùng chờ kết quả đó
let refreshing: Promise<string> | null = null;

export function refreshAccessToken(): Promise<string> {
  refreshing ??= axios
    .post<{ access_token: string }>('/api/auth/refresh', null, { withCredentials: true
})
    .then(unwrap)
    .then((res) => {
      setAccessToken(res.data.access_token);
      return res.data.access_token;
    })
    .finally(() => {
      refreshing = null;
    });
  return refreshing;
}

http.interceptors.response.use(
  unwrap,
  async (error: AxiosError) => {
    const original = error.config as RetriableConfig | undefined;
    const isAuthCall = original?.url?.startsWith('/auth/');

    if (error.response?.status !== 401 || !original || original._retried || isAuthCall)
{
      throw error;
    }

    original._retried = true; // chặn vòng lặp vô hạn
    try {
      await refreshAccessToken();
      return await http(original); // interceptor request gắn token mới
    } catch (refreshError) {
      setAccessToken(null);
      onSessionExpired();
      throw refreshError;
    }
  },
);
```

Ba điểm dễ sai:

- **Chỉ refresh một lần:** nhiều request cùng nhận 401 phải dùng chung một lần refresh. Nếu mỗi request tự refresh, token xoay vòng sẽ bị coi là dùng lại và cả phiên bị thu hồi.
- **Chặn vòng lặp:** cờ `_retried` để một request chỉ thử lại một lần.
- **Không refresh cho chính `/auth/*`:** đăng nhập sai mà cũng refresh là vô lý.

17.4 AuthContext

```
interface User {
  id: number;
  email: string;
}
interface AuthValue {
  user: User | null;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<void>;
  logout: () => Promise<void>;
}

const AuthContext = createContext<AuthValue | null>(null);

export function AuthProvider({ children }: { children: ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [isLoading, setIsLoading] = useState(true);

  useEffect(() => {
    setSessionExpiredHandler(() => setUser(null));
    // F5 làm mất access token trong bộ nhớ: thử lấy lại bằng refresh token trong
cookie
    refreshAccessToken()
      .then(() => http.get<User>('/auth/me'))
      .then((res) => setUser(res.data))
      .catch(() => setUser(null))
      .finally(() => setIsLoading(false));
  }, []);

  const value = useMemo<AuthValue>(
    () => ({
      user,
      isLoading,
      login: async (email, password) => {
        const res = await http.post<{ access_token: string }>('/auth/login', { email,
password });
        setAccessToken(res.data.access_token);
        const me = await http.get<User>('/auth/me');
        setUser(me.data);
      },
      logout: async () => {
        await http.post('/auth/logout').catch(() => undefined);
        setAccessToken(null);
        setUser(null);
      },
    }),
    [user, isLoading],
  );

  return <AuthContext value={value}>{children}</AuthContext>;
}
```

```
export function useAuth() {
  const value = use(AuthContext);
  if (!value) throw new Error('useAuth phải nằm trong AuthProvider');
  return value;
}
```

17.5 Protected route

```
export function ProtectedRoute() {
  const { user, isLoading } = useAuth();
  const location = useLocation();

  if (isLoading) return <p>Đang kiểm tra đăng nhập...</p>; // chưa biết thì đừng đá về
/login
  if (!user) return <Navigate to="/login" replace state={{ from: location.pathname }}
/>;
  return <Outlet />;
}
```

Cách gắn vào router xem đoạn `App` ở mục 15.1: bọc các trang cần đăng nhập trong `<Route element={<ProtectedRoute />}>`.

`isLoading` rất quan trọng: nếu thiếu, người đã đăng nhập bấm F5 sẽ bị đá về `/login` trong tích tắc. Sau khi đăng nhập, gọi `navigate(location.state?.from ?? '/', { replace: true })` để quay về trang đang định vào.

> **Protected route chỉ là trải nghiệm, không phải bảo mật.** Mã JavaScript nằm ở trình duyệt, ai cũng sửa được. Quyền thật phải kiểm tra ở server trên mỗi request.

17.6 Nối với API: proxy và cookie path

```
// vite.config.ts
export default defineConfig({
  plugins: [react()],
  server: { proxy: { '/api': { target: 'http://localhost:3000', changeOrigin: true } }
},
});
```

Proxy làm trình duyệt coi API và trang web là **cùng origin**, nên không gặp CORS và cookie hoạt động đơn giản hơn.

> **Đã chạy bằng trình duyệt thật với API NestJS thật qua proxy Vite:**
>
> - Chưa đăng nhập vào `/` thì sang `/login`. Đăng nhập xong về `/`, hiện đúng email.
> - Cookie `HttpOnly`, `SameSite=Lax`, JavaScript không đọc được, `localStorage` rỗng.
> - Bấm F5: gọi `POST /api/auth/refresh` 200, rồi `GET /api/auth/me` 200, vẫn ở `/`.
> - **Cookie sai path:** server đặt `Path=/auth` trong khi trình duyệt gọi `/api/auth/...`. Đăng nhập vẫn thành công, nhưng F5 thì cookie không được gửi: `refresh` trả 401 và người dùng văng ra `/login`. Lỗi này rất khó nhận ra. `Path` của cookie phải khớp URL **trình duyệt nhìn thấy**.

Việc cần làm:

1. Tạo `lib/http.ts`, chuyển toàn bộ lời gọi API sang dùng `http`.
2. Làm trang đăng nhập, `AuthProvider`, `ProtectedRoute`.
3. Nối với API NestJS Part 27. Xác nhận trong DevTools: cookie có `HttpOnly`, `localStorage` trống.
4. Đặt thời hạn access token 20 giây để thử: gọi API sau 20 giây vẫn thành công, và Network có đúng một `refresh`.
5. Mở hai request cùng lúc khi token hết hạn, xác nhận chỉ có một `refresh`.
6. Xoá dòng token trong database rồi thao tác, xác nhận app về `/login` và không lặp vô hạn.

PART 18 — ĐI LÀM

State toàn cục: Zustand và Redux Toolkit

Mục tiêu: Biết chọn đúng công cụ cho từng loại state, đọc được code Redux ở dự án có sẵn.

18.1 Chọn công cụ theo loại state

Loại state	Ví dụ	Dùng
Server state	Danh sách job, thông tin user	TanStack Query
URL state	Trang, bộ lọc	`useSearchParams`
Form state	Giá trị đang nhập	React Hook Form
Local state	Mở hay đóng modal	`useState`
Global client state	Giỏ hàng, giao diện sáng tối	Context, Zustand, Redux Toolkit

Thực tế, sau khi TanStack Query lo dữ liệu từ server, phần còn lại cần state toàn cục thường **rất ít**. Đừng đưa dữ liệu API vào Redux nếu không có lý do.

18.2 Zustand

```
interface CartItem {
  id: number;
  price: number;
  qty: number;
}
interface CartState {
  items: CartItem[];
  add: (item: Omit<CartItem, 'qty'>) => void;
  remove: (id: number) => void;
  clear: () => void;
}

export const useCart = create<CartState>()(
  persist(
    (set) => ({
      items: [],
      add: (item) =>
        set((state) => {
          const found = state.items.some((i) => i.id === item.id);
          return {
            items: found
              ? state.items.map((i) => (i.id === item.id ? { ...i, qty: i.qty + 1 } :
i))
              : [...state.items, { ...item, qty: 1 }],
          };
        }),
      remove: (id) => set((state) => ({ items: state.items.filter((i) => i.id !== id)
})),
      clear: () => set({ items: [] }),
    }),
    { name: 'cart' }, // tự lưu vào localStorage với khoá "cart"
  ),
);

// Selector: component chỉ render lại khi đúng phần này đổi
export const selectTotal = (state: CartState) => state.items.reduce((s, i) => s +
i.price * i.qty, 0);
```

```
// Trong component: chỉ render lại khi total đổi
const total = useCart(selectTotal);
const add = useCart((state) => state.add);
```

18.3 Redux Toolkit

```
interface CartItem {
  id: number;
  price: number;
  qty: number;
}

const cartSlice = createSlice({
  name: 'cart',
  initialState: { items: [] as CartItem[] },
  reducers: {
    added(state, action: PayloadAction<Omit<CartItem, 'qty'>>) {
      const found = state.items.find((i) => i.id === action.payload.id);
      if (found) found.qty += 1; // trông như sửa trực tiếp, Immer tạo state mới phía
sau
      else state.items.push({ ...action.payload, qty: 1 });
    },
    removed(state, action: PayloadAction<number>) {
      state.items = state.items.filter((i) => i.id !== action.payload);
    },
  },
});

export const { added, removed } = cartSlice.actions;

export const store = configureStore({ reducer: { cart: cartSlice.reducer } });

export type RootState = ReturnType<typeof store.getState>;
export type AppDispatch = typeof store.dispatch;
export const useAppDispatch = useDispatch.withTypes<AppDispatch>();
export const useAppSelector = useSelector.withTypes<RootState>();
```

```
// main.tsx bọc <Provider store={store}>, sau đó trong component:
const items = useAppSelector((state) => state.cart.items);
const dispatch = useAppDispatch();
dispatch(added({ id: 1, price: 100 }));
```

18.4 So sánh

	Context	Zustand	Redux Toolkit
Cài thêm	Không	Rất nhẹ	Hai gói
Lượng code	Ít	Ít nhất	Nhiều hơn
Render lại	Mọi component dùng context	Theo selector	Theo selector
Công cụ debug	Không có	Có thể thêm	Redux DevTools rất mạnh
Hợp với	Giá trị ít đổi: theme, user	Dự án nhỏ và vừa	Dự án lớn, team đông, đã có sẵn Redux

Nhiều công ty vẫn dùng Redux trong dự án có sẵn, nên cần đọc được: action, reducer, selector, `dispatch`.

> **Việc cần làm:**
>
> 1. Làm tính năng "Lưu tin" bằng Zustand, lưu xuống `localStorage` bằng `persist`. F5 vẫn còn.
> 2. Dùng selector, thêm `console.log` trong component để chứng minh component không liên quan không render lại.
> 3. Làm lại đúng tính năng đó bằng Redux Toolkit, cài Redux DevTools để xem action.
> 4. Viết ra lý do bạn sẽ chọn công cụ nào cho dự án job listings.

PART 19 — ĐI LÀM

Next.js cơ bản và Server Components

> **Mục tiêu:** Hiểu App Router, ranh giới server và client, Server Actions. Đủ để bắt tay vào dự án Next.js có sẵn.

19.1 Khi nào cần Next.js

Cần	Chọn
Trang công khai cần SEO: bán hàng, tin tức, tuyển dụng	Next.js
Dashboard sau đăng nhập, không cần SEO	Vite + React là đủ
Đã có API Laravel hoặc NestJS riêng	Cả hai đều được, Next.js gọi API đó

```
npx create-next-app@latest
```

19.2 Định tuyến theo thư mục

File	Vai trò
`app/page.tsx`	Trang `/`
`app/jobs/[id]/page.tsx`	Trang `/jobs/123`
`app/layout.tsx`	Khung dùng chung, bắt buộc có `html` và `body`
`app/loading.tsx`	Giao diện chờ, tự bọc trong Suspense
`app/error.tsx`	Bắt lỗi của trang
`app/not-found.tsx`	Trang 404

```
export const metadata: Metadata = { title: 'Việc làm', description: 'Danh sách tin tuyển dụng' };

export default function RootLayout({ children }: { children: ReactNode }) {
  return (
    <html lang="vi">
      <body>
        <nav>Trang web việc làm</nav>
        {children}
      </body>
    </html>
  );
}
```

19.3 Server Component và Client Component

Trong thư mục `app`, **mọi component mặc định là Server Component**.

	Server Component	Client Component (`'use client'`)
Chạy ở đâu	Server	Server (render đầu) và trình duyệt
`async`, đọc database, dùng secret	Được	Không
`useState`, `useEffect`, `onClick`	Không	Được
JavaScript gửi xuống trình duyệt	Không	Có

```
export const dynamic = 'force-dynamic'; // không prerender lúc build, mỗi request render lại

// Mặc định là Server Component: được async, đọc được database, không gửi JS xuống trình duyệt
export default async function HomePage() {
  const jobs = await getJobs();
  return (
    <main>
      <ul>
        {jobs.map((job) => (
          <li key={job.id}>
            <Link href={`/jobs/${job.id}`}>{job.title}</Link>
            <LikeButton /> {/* Chỗ duy nhất cần tương tác mới là Client Component */}
          </li>
        ))}
      </ul>
      <AddJobForm />
    </main>
  );
}
```

```
'use client'; // ranh giới: file này và những gì nó import chạy được ở trình duyệt

import { useState } from 'react';

export function LikeButton() {
  const [liked, setLiked] = useState(false);
  return <button onClick={() => setLiked(!liked)}>{liked ? 'Đã thích' : 'Thích'}
</button>;
}
```

Nguyên tắc: đặt `'use client'` thấp nhất có thể trong cây component. Chỉ nút cần tương tác mới là client, phần còn lại vẫn ở server.

19.4 Route động và SEO

```
interface Props {
  params: Promise<{ id: string }>; // từ Next 15, params là Promise
}

export async function generateMetadata({ params }: Props): Promise<Metadata> {
  const { id } = await params;
  const job = await getJob(Number(id));
  return { title: job?.title ?? 'Không tìm thấy' }; // <title> riêng cho SEO
}

export default async function JobPage({ params }: Props) {
  const { id } = await params;
  const job = await getJob(Number(id));
  if (!job) notFound(); // hiển thị not-found, trả về 404
  return <h1>{job.title}</h1>;
}
```

> **Đã chạy thử trên Next 16.4:** `/jobs/2` có `title` riêng. Nhưng `notFound()` **không phải lúc nào cũng trả HTTP 404**: khi có `app/loading.tsx`, `/jobs/99` trả về **200**, vì response đã bắt đầu stream trước khi biết không có dữ liệu. Bỏ `loading.tsx` thì trả đúng **404**. Trang cần SEO chuẩn nên lưu ý điểm này.

19.5 Server Actions

```
'use server'; // mỗi hàm export ở đây là một endpoint công khai

import { revalidatePath } from 'next/cache';
import { addJob } from './lib/jobs';

export interface AddJobState {
  error?: string;
  ok?: boolean;
}

export async function addJobAction(_prev: AddJobState, formData: FormData):
Promise<AddJobState> {
  const title = String(formData.get('title') ?? '').trim();
  if (title.length < 3) return { error: 'Tiêu đề tối thiểu 3 ký tự' }; // luôn validate
lại ở server
  await addJob(title);
  revalidatePath('/'); // làm mới dữ liệu của trang chủ
  return { ok: true };
}
```

```
'use client';

import { useActionState } from 'react';
import { addJobAction } from './actions';

export function AddJobForm() {
  const [state, formAction, isPending] = useActionState(addJobAction, {});
  return (
    <form action={formAction}>
      <label htmlFor="title">Tiêu đề</label>
      <input id="title" name="title" />
      {state.error && <p role="alert">{state.error}</p>}
      <button disabled={isPending}>{isPending ? 'Đang lưu...' : 'Thêm'}</button>
    </form>
  );
}
```

Mỗi Server Action là một endpoint công khai, ai cũng gọi được, kể cả không qua form của bạn. Phải kiểm tra đăng nhập, quyền và validate như một API thật. Đồng thời luôn cập nhật bản vá của `next` và `react`: cuối năm 2025 đã có lỗ hổng nghiêm trọng trong React Server Components.

19.6 Trạng thái chờ và lỗi

```
export default function Loading() {
  return <p>Đang tải...</p>; // tự bọc page trong Suspense
}
```

```
'use client'; // error boundary bắt buộc là Client Component

export default function ErrorPage({ error, reset }: { error: Error; reset: () => void
}) {
  return (
    <div role="alert">
      <p>Có lỗi: {error.message}</p>
      <button onClick={reset}>Thử lại</button>
    </div>
  );
}
```

Đã chạy thử qua HTTP thật: Server Component render dữ liệu ngay trong HTML (tốt cho SEO). Gửi form Server Action với tiêu đề ngắn thì hiện lỗi validate; tiêu đề hợp lệ thì thêm được và danh sách cập nhật. Phần cache nâng cao của Next.js thay đổi khá nhiều giữa các bản nên không đưa vào đây: khi cần, hãy đọc docs của đúng bản bạn đang dùng.

Việc cần làm:

1. Tạo app Next.js, làm trang danh sách job và trang chi tiết, gọi API Laravel hoặc NestJS trong Server Component.
2. Thêm `generateMetadata`. Xem nguồn trang (View Source) để xác nhận có sẵn nội dung và `title`.
3. Làm nút "Lưu tin" là Client Component, phần còn lại vẫn là Server Component.
4. Làm form thêm job bằng Server Action có validate.
5. Thêm `loading.tsx`, `error.tsx`, `not-found.tsx`. Kiểm tra status code của trang không tồn tại bằng `curl -I`, có và không có `loading.tsx`.

PART 20 — ĐI LÀM

Code splitting, accessibility và checklist trước khi deploy

Mục tiêu: Làm app tải nhanh hơn, dùng được cho nhiều người hơn, và đưa lên production đúng cách.

20.1 Code splitting

```
// Mỗi trang là một file JS riêng, chỉ tải khi người dùng vào trang đó
const JobsPage = lazy(() => import('./jobs/JobsPage').then((m) => ({ default:
m.JobsPage })));
const JobForm = lazy(() => import('./jobs/JobForm').then((m) => ({ default: m.JobForm
})));
```

`lazy` cần một module có `export default`. Nếu component dùng named export thì đổi như ví dụ trên. Component `lazy` phải nằm trong `<Suspense>`.

Đã kiểm chứng bằng `vite build`: `JobsPage` (0,93 kB) và `JobForm` (122,9 kB, gồm React Hook Form và Zod) được tách thành file riêng, chỉ tải khi vào đúng trang. File chính 347 kB, trong đó có React, router và TanStack Query.

20.2 Accessibility cơ bản

Làm	Tránh
`button` cho hành động, `a` cho chuyển trang	`div onClick` (không dùng được bằng bàn phím)
`label` cho mọi ô nhập	Chỉ dùng `placeholder`
`alt` có ý nghĩa cho ảnh	Bỏ trống `alt` với ảnh có nội dung
Thẻ ngữ nghĩa: `nav`, `main`, `article`, `h1`–`h3` đúng thứ tự	Toàn `div`
Dùng Tab đi được khắp trang, thấy rõ ô đang chọn	`outline: none` mà không có thay thế
Thông báo lỗi có `role="alert"`	Lỗi chỉ thể hiện bằng màu đỏ

20.3 Checklist trước khi deploy

Hạng mục	Kiểm tra
Chất lượng	`tsc --noEmit`, lint, test đều qua trong CI
Biến môi trường	Chỉ biến `VITE_*` lộ ra trình duyệt: **không để secret ở đó**
SPA routing	Server trả `index.html` cho mọi đường dẫn, nếu không F5 ở `/jobs/1` sẽ 404
API	CORS đúng domain, cookie có `Secure` trên HTTPS
Lỗi	Có Error Boundary, gắn công cụ theo dõi lỗi như Sentry
Tốc độ	Chạy Lighthouse, ảnh đúng kích thước, kiểm tra kích thước bundle

Việc cần làm:

1. Tách mỗi trang thành một chunk. So sánh kích thước file chính trước và sau.
2. Dùng dự án chỉ bằng bàn phím, sửa các chỗ không dùng được.
3. Chạy Lighthouse, đưa điểm Accessibility và Performance lên từ 90.
4. Deploy lên Vercel hoặc Netlify. Mở thẳng `/jobs/1` và bấm F5 để kiểm tra.

— Hết tài liệu — Quay lại chat khi bạn xong mỗi Part để được review & giải thích tiếp phần sau —